



SOFTWARE REQUIREMENTS SPECIFICATION

MIS Consulting



Catalyst House, 720 Centennial Court,
Centennial Park, Elstree, Herts, London, WD6 3SY
M: +44 (0) 870 850 0470
E: info@msbcgroup.com



Document Control:

Date	Version	Description	Drafted By	Reviewed By
03 July 2025	1.0	Initial Requirements & Functional Scope	Vedant Pandya Business Analyst	Shakti Gohil HOD Software

Commented [SA1]: to be updated to the date this document will be sent out

Commented [SA2]: update to my details

Table of Contents

1. Introduction	5
1.1 Document Purpose	5
1.2 Project Purpose	5
1.3 Project Objectives.....	5
1.4 Project Stakeholders.....	5
1.5 Assumptions.....	6
1.6 Acronyms and Abbreviations.....	7
2. User Personas	7
2.1 Student.....	7
2.2 Teacher / Educator.....	7
2.3 School Leadership Team.....	8
2.4 Admissions & Marketing Officer.....	8
2.5 School Inspector.....	8
3. Scope	8
4. Sequence Diagram.....	10
5. Functional Requirements – General	10
6. Functional Requirements – Use Case 1: School Inspections	13
6.1 Start New Inspection	13
6.2 Upload Previous Reports / Evidence	14
6.3 Generate Report Based on KHDA Framework	15
6.4 Ask Questions About Generated Report.....	16
6.5 Customize Report Format.....	17
6.6 Export / Download Report	18
6.7 Reports Library	19
7. Functional Requirements – Use Case 2: Curriculum & Lesson Planning.....	21
7.1 Curriculum Content Repository	21
7.2 Curriculum Design & Lesson Plan Generator	22
7.3 Personalized Learning Pathways for Students	23
7.4 Collaboration & Resource Sharing for Educators.....	25
7.5 Classroom Scheduling & Resource Allocation.....	26
7.6 Student Progress Tracking & Analytics.....	27
7.7 Content Recommendation Engine	29
7.8 Assessment & Evaluation Tools	30

7.9	Lesson Plan Modification & Adaptation.....	32
7.10	Audit & Version Control for Curriculum Changes	33
8.	Functional Requirements – Use Case 3: Student Learning Assistant	35
8.1	Student Learning Assistant	35
8.2	Student Query Input & Recommendations	36
8.3	Homework & Assignment Support.....	37
8.4	Learning Persona & Goal Selector.....	38
8.5	Upload Classroom Materials for Context.....	39
8.6	Progress Tracker & Feedback Summary	40
8.7	Student Guardrails & Prompt Moderation	41
8.8	Teacher Control Console.....	42
9.	Functional Requirements – Use Case 4: Admissions & Marketing	44
9.1	Admissions Process Assistant.....	44
9.2	Applicant Profile Evaluation.....	45
9.3	Admissions Insights Dashboard	46
9.4	Enquiry Management & Follow-Up Generator.....	47
9.5	Marketing Campaign Content Generator	48
9.6	Campaign Performance Insights.....	49
10.	Non-Functional Requirements.....	51
11.	System Requirements.....	53
11.1	User Interface	53
11.2	Database.....	53
11.3	Communication/Security	54
11.4	System Integration.....	55
11.5	Constraints and Limitation	55
11.6	Legal	56
11.6.1	Data Protection and Privacy.....	56
11.6.2	AI Content and Educational Use.....	56
11.6.3	Liability Limitation.....	56
12.	System Architecture	57
12.1	Overview	57
12.2	AI Subsystems	57
12.2.1	Router & Orchestration Layer	58
12.2.2	Knowledge-Centric Q&A Route.....	58

12.2.3	Curriculum & Lesson-Plan Composer	58
12.2.4	Analytical Report & Code-Generation Agent	59
12.2.5	Vision & Inspection Agent	59
12.2.6	Retrieval Infrastructure (shared by all routes)	59
12.2.7	Fallback & Governance Logic	60
12.3	N-tier Architecture	60
12.3.1	Presentation Component	60
12.3.2	Business Logic Component	60
12.3.3	Data Access Component	61
12.3.4	Communication & Security Component	61
12.3.5	API Component	61
12.4	Microservices Architecture	63
12.4.1	Key Benefits	64
12.4.2	Core Microservices and Their Responsibilities	64
12.4.3	Communication and Integration Pattern	66
12.4.4	Security and Compliance (Offline-Oriented)	66
12.5	Method of Development	67
12.6	High-Level Technical Design	68
13.	Hardware Requirements	69
14.	Acceptance Criteria	69
15.	Product Future	72
16.	Open Queries	73

1. Introduction

1.1 Document Purpose

The purpose of this document is to outline the comprehensive requirements of the proposed AI-Driven Educational Platform. This document serves as a formal agreement between stakeholders, ensuring a shared understanding of the platform's objectives, features, user roles, and technical expectations. It aims to guide the development lifecycle, facilitate validation and verification processes, and serve as a reference point for scope management, change tracking, and quality assurance activities.

1.2 Project Purpose

The primary purpose of this project is to design and develop an AI-enabled platform tailored for the education sector in the UAE. The platform is envisioned to empower schools and educational authorities by integrating intelligent automation into their core functions, ranging from inspections and curriculum planning to student support and admissions. By harnessing the potential of AI technologies, the platform aims to improve the consistency, accuracy, and efficiency of academic and administrative processes, ultimately contributing to higher educational standards, increased student engagement, and data-informed decision-making across institutions.

1.3 Project Objectives

The key objectives of the project include:

- **Automate the School Inspection Process:** Introduce an AI-driven inspection framework that standardises evaluations, minimises subjectivity, and provides actionable insights to improve institutional performance.
- **Enhance Curriculum and Lesson Planning:** Assist educators in creating curriculum-aligned lesson plans, homework, quizzes, and other instructional content through AI recommendations and templates, saving time and increasing instructional quality.
- **Support Personalised Student Learning:** Offer AI-powered tools that help students learn at their own pace by delivering tailored study plans, tracking academic progress, and recommending targeted learning resources based on individual performance.
- **Streamline Admissions and Boost Marketing:** Use AI algorithms to optimise admissions workflows, identify high-potential applicants, and enhance digital marketing efforts to improve enrolment rates and outreach effectiveness.

1.4 Project Stakeholders

Role	Resource	Email Id
Sponsor	Riza Kaya	riza@kayamiconsulting.com
Head Of Development	Shakti Gohil	shakti.gohil@msbcgroup.com
AI Solution Lead	Suhail Ahmed	suhail.ahmed@msbcgroup.com
Junior AI/ML Engineer	Sahil Sharma	sahil.sharma@msbcgroup.com
Junior AI/ML Engineer	Rohit Jangid	rohit.jangid@msbcgroup.com
Sr. UI/UX Design	Aesha Thakkar	aesha.thakkar@msbcgroup.com
UI/UX Designer	Ishika Anandani	ishika.anandani@msbcgroup.com

Business Analyst	Ajay Chakrobarty	ajay.chakrobarty@msbcgroup.com
Business Analyst	Vedant Pandya	vedant.pandya@msbcgroup.com

1.5 Assumptions

- The entire system will be deployed on-premises within school infrastructure, ~~due to data privacy mandates in Dubai.~~
- The system is expected to function offline, with a file-based or intranet-based sync mechanism to handle content and data ingestion.
- Initial deployments will be handled manually per school.
- The primary user experience will be conversational, akin to ChatGPT or Claude, with use-case-specific suggestions and follow-ups.
- ~~Multi-step tasks will be handled in purely conversational flows rather than hybrid UI modals or sidebars.~~
- Voice input and AI-generated audio responses will be available across modules and languages.
- There are five primary personas: Student, Teacher, School Leadership Team, Admissions & Marketing Officer and School Inspector. Each persona will have role-based access to specific modules and views. These permissions will be centrally managed by school-level Admins.
- Students will interact with the system only under supervision, within classroom settings; unsupervised or remote student access is out of scope.
- The system will use four dedicated models, **each fine-tuned** per use case, plus one General Orchestrator Model to route queries.
- If a query touches multiple use cases, the system will support model chaining and intelligently hand over the request internally.
- Unsupported or irrelevant queries will result in fallback responses or redirection to valid modules, not system failure.
- All content responses will be based on pre-vetted, internal knowledgebases, including lesson materials and institutional guidelines.
- External web content will only be allowed if part of the school's internal uploaded database.
- The system will support bilingual usage (English and Arabic) across all user-facing modules.**
- Users will toggle their preferred language via a button in the interface, and system content will reflect that language in both prompts and outputs.
- No automatic language detection or translation during conversations will occur; preference is manually set.
- Integration with existing Student Information Systems (SIS) will occur via a mapping setup during implementation.
- No real-time third-party integrations (e.g., Google Analytics, Power BI) are expected; all analytics will be locally built and rendered.
- There is no integration planned with live social media platforms; marketing content will be generated and downloaded manually for upload.
- Inspection reports and other outputs can be edited for content and values, including graphs (except graph types).
- Real-time collaboration on reports is not allowed; only one user at a time can view/edit based on their access.

Commented [SA3]: we have the canvas flow for social media posts, that would be a hybrid UI

Commented [SA4]: one model per use case, finetuning is not required for this project

Commented [SA5]: bilingual will be supported for text, but for speech to text - only English will be supported

- All generated content, conversations, and edits will be logged in audit trails for compliance and review.
- Reports can be exported only as PDFs; no DOCX, XLSX, or collaborative formats.
- The system will include guardrails for students, including prompt filtering, audit logging, and AI behaviour pre-configuration.
- Teachers will retain control over what content students can upload, view, or receive recommendations from.
- The platform will use specialised LLM prompts per use case, ensuring responses are context-sensitive and age-appropriate.

1.6 Acronyms and Abbreviations

Key	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
SIS	Student Information System
KHDA	Knowledge and Human Development Authority (Dubai's education regulator)
NIM	NVIDIA Inference Microservice
API	Application Programming Interface
RAG	Retrieval Augmented Generation

2. User Personas

2.1 Student

The student is a supervised end-user who interacts with the platform for academic support, content clarification, and personalised guidance. Students will engage with the system primarily within the classroom, under teacher supervision, and will have access to features such as uploading study materials, receiving feedback, and tracking learning progress. The platform will maintain an audit trail of student interactions to ensure compliance with safeguarding policies. The assistant is designed to support students without enabling unsupervised internet access, and all content is served from a school-approved internal knowledgebase. Key challenges addressed include difficulty in accessing personalised support, reliance on static textbooks, and limited digital interactivity.

2.2 Teacher / Educator

The Teacher or Educator is a central user of the platform, responsible for designing and delivering instructional content to students. The platform will assist educators in creating curriculum-aligned lesson plans, homework, quizzes, and supplementary materials, all guided by AI-generated recommendations. Teachers will also benefit from the ability to track student performance and engagement issues. The platform is designed to alleviate the common burdens faced by teachers, such as time constraints, repetitive planning tasks, and the difficulty of catering to varied learning needs within the classroom.

2.3 School Leadership Team

Including roles such as Principal, Vice Principal, or Heads of Department, are tasked with overseeing daily operations, maintaining academic standards, and facilitating continuous

improvement. This user group depends on the platform to monitor school-wide performance indicators, analyse inspection feedback, and support strategic planning. Additionally, the leadership team will leverage AI capabilities to assist teachers with curriculum planning and identify areas requiring intervention. Their current challenges include administrative overload and a lack of real-time, actionable insights for decision-making.

2.4 Admissions & Marketing Officer

The Admissions and Marketing Officer plays a crucial role in student enrolment and institutional growth. This user is responsible for managing the admissions funnel, nurturing prospective leads, and executing marketing campaigns. The AI capabilities within the platform will enable them to automate communication workflows, predict enrolment trends, and optimise campaign targeting based on data analytics. Their challenges often include low visibility into applicant behaviour, inefficiencies in lead conversion, and reliance on manual tools for campaign management, all of which the platform is designed to resolve.

2.5 School Inspector

Typically, a representative from a regulatory body or education authority, is responsible for conducting evaluations and audits of schools. This persona requires access to AI-generated inspection reports, and standardised performance metrics. Their primary goal is to ensure institutional compliance with national education standards while reducing subjectivity in assessments. Traditionally, manual inspection processes have been time-intensive and inconsistent, which the platform seeks to address through automation and data-driven insights.

3. Scope

Module	Sub Module	Features
General		
Conversational Interface	UI Interface	Chat-first interface, use case routing, toggle language, multi-turn support
	Use Case Selector	Dropdown / icon-based selection of modules
Prompting & Actions	Prompt Suggestions & Quick Actions	Contextual suggestions, recent query memory, adaptive prompting
	Voice Input	Voice-to-text, multilingual support (English/Arabic)
File Handling	File Upload & Ingestion	Upload PDFs, DOCX, CSV; process for context or grounding
User Management	Role & Permission Control	School-based RBAC
	Access Management	Role-specific module access and view restrictions
Knowledge Sync	Offline Sync Mechanism	File-based local sync; fallback from intranet to device storage
Logging & History	History & Saved Outputs	View past queries, regenerate, export results
AI Orchestration	Multi-modal Routing	Route to correct LLMs, fallback handling, model chaining
Use Case 1: School Inspection		
	Start New Inspection	Preconfigured framework, conversational entry

Commented [SA6]: bilingual

Inspection Flow	Upload Previous Reports / Evidence	PDFs, performance files, visual uploads for analysis
Report Generation	Generate Report based on Framework	Canvas-style generation, voice summarisation and editing of textual values
	Ask Questions about report	Q&A with generated content, explanation, refinement
	Customize Report Format	Section rearrangement, priority highlights, edit capabilities
Output Management	Export Report	PDF export with template-based formatting
	Role-Based Viewing	Library access as per the role
Use Case 2: Curriculum & Lesson Planning		
Content & Planning	Curriculum Content Repository	Upload, tag, and manage lesson plans, materials
	Curriculum Design & Lesson Generator	AI-assisted planning with alignment to national standards
Personalization & Support	Personalization Learning Pathways	Student-based curriculum branching, pre-defined learning persona
Scheduling & Analytics	Classroom Scheduling & Resource Planner	Suggest optimal time slots, resource usage summaries
	Student Progress Tracker	Visual Insights, heatmaps, scoring analysis
Use Case 3: Student Learning Assistant		
Student Interaction	Student Learning Assistant	Guided conversation agent for Q&A and studying
	Student Query Input & Recommendations	Intent detection, follow-up material suggestions
Content & Context	Upload Classroom Materials	Student upload with teacher approved ingestion
	Progress Tracker & Feedback	Session-based summaries, goal alignment
Safeguards & Oversight	Prompt Moderation	Predefined guardrails, offensive prompt filtering
Use Case 4: Admissions & Marketing		
Admissions Automation	Admission Process Assistant	Conversational querying for intake, forms and eligibility
	Applicant Profile Evaluation	Profile Evaluation based on admission parameters
Insights & Follow-Up	Admission Insights Dashboard	Funnel admission analytics and trends
	Enquiry Management	Follow-up generator, lead scoring, draft replies
Campaign Generation	Marketing Campaign Generator	Platform specific post generator, picture editing (text and values)
	Campaign Performance Insights	Campaign level impact breakdown and reach analytics

Commented [SA7]: the system will not undertake any scheduling tasks, out of scope. unclear on what resource usage summaries mean, however it will assist with creating lesson plans

Data Forecasting	Enrolment Forecast & Churn Prediction	Predictive insights across inspection, curriculum, and student data, trends and dropout risks
------------------	---------------------------------------	---

Commented [SA8]: forecasting is out of scope, we will display existing churn rate, however we will not predict churn rates - same for enrolments

4. Sequence Diagram



System Overview.png



Use Case 1.png



Use Case 2.png



Use Case 3.png



Use Case 4.png

5. Functional Requirements – General

These requirements define the core platform-level functionality common across all use cases. They establish the foundational experience of interacting with the system via a conversational interface, including use case selection, prompt suggestions, voice input, file ingestion, offline sync, access management, and model orchestration.

- The system shall present a single unified conversational interface for all users regardless of use case.
- User input (text/voice) shall be entered into a chat interface similar to Claude/ChatGPT.
- The UI shall support multi-step conversations, where AI prompts the user progressively (e.g., “Now upload your syllabus file”).
- The system shall support offline operations using local compute and file-based sync for context ingestion.
- The interface shall offer a selector or configurator to choose one of the four use cases:
 - School Inspections**
 - Curriculum & Lesson Planning**
 - Student Learning Assistant**
 - Admissions & Marketing**
- Once selected, the assistant will route all subsequent prompts to that use case's corresponding LLM.
- If the user submits a query involving multiple use cases, the platform shall invoke model chaining via a ~~general~~ routing model.
- Based on the active use case, the system shall show contextual prompt suggestions (e.g., “Generate lesson plan for Grade 3 Science”).
- These prompts may be triggered via ~~voice or~~ quick action button.
- ~~Suggestions shall adapt based on recent user interactions, role, and prior history.~~
- The system shall allow voice-based input using device microphone access.
- Voice input will be transcribed before submission.
- For reports and long-form responses, the assistant shall offer a “Listen to Report” feature with text-to-speech playback.
- For the various Use Cases, users shall be able to upload documents like:
 - Previous reports
 - Lesson plans
 - Curriculum documents
 - Classroom materials
- Uploads will occur within the conversation, guided by the AI (e.g., “Upload the student’s last progress report”).

- Files will be processed using offline, file-based ingestion, with metadata extraction and tagging.
- Uploaded content shall be limited to internal, vetted materials as controlled by the school administrator.
- User Roles include:
 - a. Administrator
 - b. Teacher / Educator
 - c. School Inspector
 - d. Admissions Officer
 - e. Student (limited access)
- The system shall control visibility and access per role.
- Profiles shall store language preference (English/Arabic) with toggle available on the UI.
- Only authorized roles can access sensitive actions like editing reports, managing content, or generating enrolment forecasts.
- All queries, chats, and outputs will be logged per user session.
- Users can view previous interactions, saved reports, and uploaded content.
- Multi-model conversations shall be marked and stored as a single, unified session.
- Outputs can be downloaded or saved to library (role-based visibility).
- The entire system shall be designed to function offline by default.
- Sync of system updates and ingestion of school content shall occur via:
 - a. Intranet-based file exchange
 - b. Local server mapping
 - c. No public APIs or internet-based syncs shall be used.
 - d. Every interaction (prompt, response, upload, download) shall be logged with timestamp and user ID.
- Audit trail shall include:
 - a. Content source (e.g., uploaded file, student record)
 - b. Model(s) invoked (single or chained)
- Admins shall have access to view logs filtered by user, module, or timeframe.
- The system shall include a general routing model that determines which of the 4 LLMs should respond to a given user query.
- In case of multi-use-case queries, the routing model shall perform model chaining and consolidate the outputs into a unified response.
- If the query is out-of-scope (e.g., non-educational), a fallback message like *"This platform is designed for educational use cases"* will be returned.
- All orchestrated responses shall preserve context lineage and display involved models in metadata.
- Users shall have a toggle to switch between English and Arabic at any point.
- Conversational language used by the assistant shall default based on user profile setting.
- The assistant shall not auto-detect, or switch languages based on query; language is profile-bound.
- AI responses must be generated using internal, pre-vetted content only. No internet browsing or external source calls (like Bing search, YouTube search) will be used. However, YouTube or any external links included in the internal school knowledge base may be referenced if pre-approved.
- Admins will be responsible for maintaining, tagging, and curating the internal KB.
- For generated outputs like, Inspection reports, Lesson plans and Campaign visuals, Users will be allowed to edit:
 - a. Text content

- b. Colours and labels of graphs
 - c. Fonts and visual formatting
- Graph types and structural changes will not be editable.
- All analytics, visualizations, and performance summaries shall be native to the platform. No integration with external tools (e.g., Power BI, GA) is planned.
- Dashboards will be configurable based on role and module.
- Platform must comply with Dubai data privacy and educational data protection laws for the UAE schools deployment.
- All data remains on-premises, including logs, models, and student records. No cloud services or SaaS dependencies shall exist.'
- Users with admin access should be able to:
 - a. View all registered users across roles.
 - b. Create or delete user accounts manually.
 - c. Assign, modify or revoke roles.
 - d. Define default role-based permissions.
 - e. Trigger manual sync from CSV or local school SIS (If integrated).
- Users will also be able to manage the branding configuration accessible via Configuration section based on their role. They will be able to manage settings like:
 - a. Upload institutional logo (PNG/JPG/SVG)
 - b. Define brand colour palette used across reports, posters, and dashboards.
 - c. Configure report header/footer with institution name, address and watermark.
- Admin users can access Knowledgebase management via configuration that should allow them to:
 - a. Upload and tag documents to be used in system-wide knowledgebase.
 - b. Configure metadata for documents.
 - c. Enable/Disable certain knowledgebases from being used in specific modules.
 - d. View the sync status of uploaded documents.
- Admin users can access Usage Analytics that is dashboard interface of the system to view the system-wide analytics like:
 - a. Number of queries per module
 - b. Top-used documents.
 - c. User level engagement (queries/sessions)
 - d. Filter by user roles
 - e. Export usage reports (csv)
- Admin users access Flagged Responses to view responses flagged by users. In this module they will be able to view details like:
 - a. View all responses flagged by users.
 - b. Filter by reason (incorrect, inappropriate, irrelevant)
 - c. Each flagged entry displays:
 - i. Username
 - ii. User Role
 - iii. Original Query + AI Response
 - iv. Reason for Flag

6. Functional Requirements – Use Case 1: School Inspections

6.1 Start New Inspection

a. Overview

This module allows users (Inspectors, School Leadership) to initiate a new AI-assisted inspection session. The system will generate an inspection baseline based on the selected KHDA (or equivalent) framework, optionally pre-fill details using SIS data, and prepare the environment for ~~interactive~~ analysis and report drafting.

b. Preconditions

- User is authenticated and has appropriate role (Inspector, Leadership Team, or Admin)
- Inspection framework templates are configured.
- SIS mapping (if any) is set up for the school instance.

c. Post Conditions

- A new inspection session ID is generated and stored.
- A working inspection environment is created.
- The system is ready to accept evidence, inputs, and questions.

d. Normal Flows

- User selects **"Start New Inspection"** from dashboard or prompt suggestions.
- System prompts for required metadata:
 - School name (auto-filled)
 - Inspection type (e.g., Initial, Mid-Year, Final)
 - Framework version (e.g., KHDA 2024)
- Optional: System auto-fetches base data from SIS (attendance, grades, past scores)
- Inspection session is initialized with:
 - Sectional breakdown (e.g., Teaching Quality, Student Outcomes, etc.)
 - Prompt areas for input.
 - Upload panel for supporting files.
- A chat-style session begins, anchored to this inspection.
- User starts asking questions, uploading data, and refining content.

e. Field Validations

- School name: Mandatory
- Inspection type: Must be one of predefined values.
- Framework version: Must match configured templates.
- Duplicate inspection for same school + period → show warning

f. Business Validation

- ~~Only one inspection in progress per school per term~~
- Edits to framework once inspection starts must be restricted to Admin.
- SIS data not available → system continues with manual entry only.
- Inspection framework file missing → system shows error and halts process.
- User disconnects mid-process → session will be terminated and will not resume automatically post login.

6.2 Upload Previous Reports / Evidence

a. Overview

This module enables users to upload previous inspection reports, school self-assessments, academic results, photos, or internal observations. The AI model uses this contextual evidence to tailor responses, highlight trends, and generate richer reports.

b. Preconditions

- A new or existing inspection session is active.
- User is authenticated and authorized for this school.
- Files are within supported format and size constraints.

c. Post Conditions

- Uploaded files are parsed and indexed.
- Extracted content is embedded into the current inspection context.
- Files are referenced in the generated reports with metadata tags.

d. Normal Flows

- User clicks "Upload Evidence" or drops files into the inspection interface.
- System accepts the files and performs:
 - Format validation
 - Content extraction (text + table parsing + OCR if needed)
 - Content summarization and tagging
- Summarized info is shown to the user for confirmation.
- AI model is updated with this contextual evidence.
- Files are stored in inspection-specific document library with metadata:
 - File Name
 - Type (e.g., Academic Report, Self-Assessment)
 - Date Uploaded
 - Parsed Summary
 - Linked Inspection Section

e. Field Validations

- Supported formats: .pdf, .docx, .pptx, .xlsx, .jpg, .png
- Max size: 15MB per file; Max 10 files
- File name must not include special characters or be empty.

f. Business Validation

- Files must be tagged to a specific inspection section or theme.
- Duplicate files (same hash) within a session should be rejected.
- Students and unauthorized users cannot ~~should not be able to~~ view the inspection module.
- Uploaded file is corrupted → show "File unreadable, please try another."

- Upload timeout due to network → allow retry.
- File content in unsupported language → flag and request manual summary.

6.3 Generate Report Based on KHDA Framework

a. Overview

This module allows users to generate a detailed inspection report aligned with a regulatory framework (e.g., KHDA) using AI. The report synthesizes uploaded evidence, SIS data, and prior interactions. The report opens in a “Canvas View” with a split-screen layout: the left side displays the structured report, and the right side is a persistent AI chat assistant for live interaction. The user can also listen to the report, regenerate sections, and share or export the report.

b. Preconditions

- Inspection session is active and has minimum required inputs (metadata, evidence, user inputs)
- Framework structure is available.
- User has appropriate role (Inspector, Leadership, Admin)

c. Post Conditions

- A structured, editable report is generated.
- The canvas interface is launched with interactive capabilities.
- Report is stored, versioned, and available for download/share.

d. Normal Flows

- User clicks “Generate Inspection Report”.
- System performs:
 - Data validation & consistency check
 - Section-wise AI summarization aligned with framework.
 - Highlighting of strengths, gaps, and improvement areas
- A new view is launched:
 - **Left Panel (Canvas Report View):**
 - Navigation by section (Teaching Quality, Governance, Wellbeing, etc.)
 - Inline editing (if user has permission)
 - Listen button to hear audio narration.
 - **Right Panel (AI Assistant):**
 - Context-aware conversation about this specific report
 - Prompt suggestions (e.g., “What are the top 3 weaknesses?”)
 - Feedback tools (e.g., thumbs up/down)
- User can:
 - Edit text, values, labels of graph within the generated report. The graph type selection is fixed, and user cannot change the chart types. For that they will have to regenerate the report.
 - Editing will happen within the same canvas and not a separate UI.
 - The chat will remain inactive on the side for Q&A during the editing.

Commented [SA9]: this covers the same flow of start new inspection. in such a case, you can remove start new inspection and keep this as this is more details. rename it to Generate Inspection Report

- Ask follow-up questions.
 - Download report (PDF)
 - Share via secure link.
- All interactions are saved in report history.

e. Field Validations

- Each section of the report must have content before download.
- Audio narration available only in English (Phase 1)
- Report versioning format: SchoolName_InspectionType_YYYYMMDD_v1.pdf.

f. Business Validation

- Only designated roles can edit or export reports.
- Voice narration cannot be generated for incomplete reports.
- Reports generated are auto tagged with audit entry and model version.
- Section data missing → system flags and blocks generation.
- Large report (>20 pages) → generation time extended, user shown loader.
- Network loss during generation → recover using autosave buffer.

6.4 Ask Questions About Generated Report

a. Overview

Once the report is generated, users can ask the AI assistant contextual questions to clarify, analyse, or explore insights from the report. This enhances usability, supports decision-making, and mimics an expert peer reviewer.

b. Preconditions

- A report is already generated and visible in the canvas view.
- The assistant chat is active and linked to the correct report session.
- User is authenticated and has viewer access to the report.

c. Post Conditions

- AI returns responses based on report content.
- Optionally suggests visual summaries or extracts sections.
- Chat history is appended to the report session's metadata.

d. Normal Flows

- User opens chat (right panel) and enters a question like:
 - "What are the 3 biggest areas of concern?"
 - "Which evidence supports the Leadership Quality section?"
- The system:
 - Anchors to the active report context
 - Queries sectional data and evidence embeddings
 - Returns a summary or respective answer.

- Additional features:
 - Voice option to listen to the response.

e. Field Validations

- ~~Max prompt size: 2000 tokens~~
- Highlighting only enabled if section mapping is available.

f. Business Validation

- Role-based filters may apply to certain sections (e.g., Governance for Admins only)
- Follow-up questions retain context only within session lifespan (e.g., 2 hours)
- User refers to a deleted section → system replies: “This section no longer exists.”
- Long chain of questions → context window may truncate earlier Q&A
- Audio not available → fallback to text-only

6.5 Customize Report Format

a. Overview

This module allows authorized users to personalize the structure, visual style, and content focus of the generated inspection report. Customization may include section ordering, inclusion/exclusion of evidence, visual themes (logos, headers), and prioritization of improvement areas. It gives schools the flexibility to align reports with their internal branding or regulatory presentation formats.

b. Preconditions

- A valid inspection report is already generated.
- User has the Admin or Inspector role.
- Editing mode is enabled.

c. Post Conditions

- A customized version of the report is stored alongside the original
- The exported/downloaded report reflects the custom selections.
- AI assistant is updated with the modified structure for contextual interaction.

d. Normal Flows

- User clicks “Customize Report Format” from the report canvas top bar.
- A side panel opens with customization options:
 - Include/exclude sections (toggle switches)
 - Reorder sections via drag-and-drop.
 - Prioritize (highlight) key sections like “Improvement Areas First”.
 - Upload custom school logo and footer.
- Optional toggles:
 - Include document titles or not.
 - Show/hide embedded evidence links.

- Include system-generated confidence score (for pilot version)
- Preview is generated live on the left canvas upon saving the changes.
- User clicks “Apply Customization” → system saves custom format as version v2.
- System offers export in new format.

e. Field Validations

- Logo file must be .png or .jpg, max 2MB.
- Cannot exclude mandatory sections (e.g., Summary, Key Findings)
- Custom section titles cannot exceed 100 characters.

f. Business Validation

- Only Inspectors or Admins can customize report format.
- Each customized format is saved with metadata (user, timestamp)
- No permanent overwrites of the original report – always kept as v1.
- User attempts to exclude all sections → system blocks action.
- Image/logo upload fails → revert to default school branding.
- Customized order breaks logic flow → system alerts with dependencies (e.g., Summary must follow Analysis)

6.6 Export / Download Report

a. Overview

This module allows users to export the finalized inspection report as a professionally formatted PDF. The PDF includes the school logo, version number, generated timestamp, section contents, and optional appendix for uploaded evidence. Download is permitted only to users with required roles, and report versions are audit-tracked.

b. Preconditions

- A report has been generated and finalized.
- User has appropriate role (Inspector, Leadership, Admin)
- Download is not restricted by policy.

c. Post Conditions

- A PDF file is generated using the current report format (customized or default)
- File is downloaded to the user’s local system.
- A download event is logged with metadata (user ID, report ID, timestamp)

d. Normal Flows

- User clicks “**Download PDF**” from top-right of the report canvas.
- System verifies:
 - All mandatory sections are completed.
 - No sync conflicts or missing data
- PDF generation process begins:

- Layout applied (customized if present)
- Content rendered with page breaks, headers, logo, footer.
- Evidence attachments (if opted-in) appended as annex.
- Download begins → PDF saved to local device.
- User is shown success message and version number.

e. Field Validations

- Report must have at least 3 main sections populated.
- ~~• PDF generation timeout: 60 seconds max~~
- Only the latest report version is downloadable unless explicitly requested.
- ~~• Downloaded PDFs include watermark.~~
- PDF generation fails due to memory constraints → system retries once, else shows error.
- Network disconnect during generation (for remote sync) → download paused.
- User tries to re-download outdated report → system prompts to regenerate or continue with archived version.

6.7 Reports Library

a. Overview

This module enables users to view, search, and access previously generated inspection reports via a centralized Report Library. Access to reports is controlled through role-based permissions at the organization/school level. Users can only see reports they are authorized to access based on their role (e.g., Inspector, Leadership, Teacher, Admin). Reports are organized by institution, date, framework type, and inspection session metadata.

b. Preconditions

- User is authenticated and belongs to an organization (school, governing body)
- At least one inspection report exists in the system.
- Role-to-access mapping is defined for the user's role within the organization.

c. Post Conditions

- User can view a filtered list of reports they are authorized to access.
- Clicking on a report opens the canvas view in read-only or editable mode (based on role)
- Access logs are maintained for auditing/reporting.

d. Normal Flows

- User navigates to **"Report Library"** from the main interface.
- System displays a grid/list of available reports, showing:
 - Report Title (e.g., "Spring 2025 - Leadership Evaluation")
 - School Name
 - Inspection Type
 - Created By
 - Status (Draft, Finalized)
 - Last Modified Date

- Access Level (Read / Edit)
- User can:
 - Apply filters (School, Type, Status, Date Range)
 - Sort reports (Newest First, Alphabetical)
 - Search by keyword or school name
- When a report is selected:
 - If the user has view-only rights → report opens in read-only mode
 - If the user has edit rights → canvas view opens in full-edit mode (Future Phase)
- A download and share button is visible only to users with sufficient permission.
- Every view/access event is stored in an audit log

e. Field Validations

- Filters must use only valid framework versions and school names
- Search input should handle fuzzy search and return contextual results
- Access Level must match RBAC policy:
 - Teacher → View only (select sections)
 - Inspector → View/Edit
 - Admin → Full Access

f. Business Validation

- Users must not be able to view reports of other institutions unless explicitly granted multi-org access.
- Sharing a report requires Admin or Inspector role.
- Only finalized reports can be downloaded.
- Reports can be archived (soft delete) but not permanently deleted without Admin override.
- Report version mismatch → system prompts user to regenerate or review older format.
- Organization role mapping changes after report creation → access updated accordingly.
- School Admin revokes access → user sees “Access Denied” page.

7. Functional Requirements – Use Case 2: Curriculum & Lesson Planning

7.1 Curriculum Content Repository

a. Overview

This module provides a centralized digital repository where educators can upload, organize, and retrieve curriculum materials such as lesson plans, worksheets, ~~multimedia content~~, quizzes, and reference documents. The repository is structured by subject, grade level, and curriculum framework, and supports role-based access, versioning, and tagging.

b. Preconditions

- User is authenticated and has Teacher, Department Head, or Admin role.
- School has been onboarded with a curriculum structure (framework uploaded or selected)

- Minimum metadata (subject, grade level) is available.

c. Post Conditions

- Content is saved, indexed, and accessible by authorized users.
- Metadata is stored for recommendation and AI indexing.
- Files are versioned and retrievable for collaboration or reuse.

d. Normal Flows

- User clicks “**Curriculum Repository**” from the main dashboard
- Repository view displays folder structure:
 - Grade Level → Subject → Topics.
- User can:
 - Upload new files (PDF, PPT, DOCX, MP4)
 - Create new entries (lesson plan, worksheet, quiz)
 - Tag content by curriculum objective, topic, grade level.
 - Set visibility (Private, School-wide, Public with moderation)
- Content is indexed with:
 - Auto-generated title (editable)
 - Description
 - Role of uploader
 - Version number
- System logs upload and assigns internal content ID.
- Other users can search, view, or download content based on permissions.

e. Field Validations

- File size limit: 100MB.
- Allowed file types: .pdf, .docx, .pptx, .xlsx, .mp4, .jpg, .png, .json
- Metadata fields (subject, grade, curriculum tag) are mandatory on upload.

f. Business Validation

- Only Teachers, Department Heads, or Admins can upload content.
- System prevents duplicate file uploads based on hash check unless versioned.
- Network disconnect during upload → ~~resume via chunking or~~ prompt retry.
- Curriculum tag missing → system prompts user before allowing upload.
- User uploads unsupported format → system rejects and shows tooltip.

7.2 Curriculum Design & Lesson Plan Generator

a. Overview

This module allows educators to generate detailed, curriculum-aligned lesson plans using AI. Teachers can input learning objectives, subject, grade level, and duration, and the system generates a structured lesson plan with suggested activities, materials, and assessments.

b. Preconditions

- User is authenticated and has Teacher or Department Head role.
- Subject and grade-level curriculum is selected or available.
- AI model for lesson planning is accessible.

c. Post Conditions

- A new lesson plan is generated and stored in the content repository.
- AI-generated plan includes structured components: objectives, activities, materials, assessments.
- User may further customize or edit the output.

d. Normal Flows

- The user initiates lesson planning by either:
 - Selecting a quick prompt like "Create a lesson plan for Grade 5 Science".
 - Typing a free-form query like "Generate an introductory lesson on photosynthesis for Grade 6 Biology".
- The system responds in the conversational thread with a request for clarification if any required details are missing. Example system prompts:
 - "What is the grade level and subject for this lesson?"
 - "Would you like this lesson to be introductory, reinforcement, or remedial?"
 - "How long should the session last?"
- The user provides inputs conversationally. The system collects:
 - Subject
 - Grade Level
 - Topic/Unit
 - Learning Objective (if not inferred from topic)
 - Duration
 - Lesson Type (Introductory / Reinforcement / Remedial)
- Once all inputs are received, the system:
 - Validates the inputs based on preconfigured curriculum constraints.
 - Routes the request to the appropriate curriculum-aligned AI model.
 - Generates and displays a structured lesson plan in the conversation, containing:
 - Title
 - Learning Objectives
 - Instructional Activities
 - Materials & Resources
 - Formative Assessment Suggestions
 - Differentiation Strategies
- The system displays actionable buttons below the generated output:
 - Edit Plan – Opens inline conversational prompts to modify specific sections.
 - Save to Repository – Saves the version under teacher's profile with metadata.
 - Export – Downloads the plan as PDF.
- Upon saving, the system:
 - Tags the lesson plan with metadata (subject, grade, topic, date).
 - Assigns a version ID for tracking changes or iterations.

e. Field Validations

- Learning objective must be 10–250 characters.
- Duration must be between 15 mins and 180 mins.
- Grade level and subject are required inputs.

Commented [SA10]: unclear on what duration is referring to

f. Business Validation

- ~~AI generation is rate limited to 10 requests/hour per user to avoid misuse.~~
- Auto-generated plans are labelled as “AI Draft” until reviewed.
- AI fails to generate due to missing information → prompt user.
- ~~Generated plan exceeds length → user prompted to reduce duration or split.~~

7.3 Personalized Learning Pathways for Students

a. Overview

This module enables educators to request AI-generated, personalized learning pathways for individual students or student groups based on academic performance, learning preferences, and curriculum goals. The system responds through a conversational interface, asking contextual questions and returning tailored learning sequences that align with both student needs and institutional objectives.

b. Preconditions

- The user must be a Teacher, Department Head, or part of the School Leadership Team with appropriate access.
- Student performance data must be available in the system (uploaded or synced from SIS).
- Curriculum structure and lesson mappings must already exist in the system.

c. Post Conditions

- A structured, stepwise learning pathway is generated.
- The user may choose to:
 - Save it to the student’s profile.
 - Share it with the student or parents (if enabled).
 - Modify or regenerate it.
- Version history and metadata are stored.

d. Normal Flows

- The teacher initiates the query through a conversational input, e.g.,
- "Create a learning plan for Ali in Grade 6 for improving his reading comprehension."
- The system parses the input and may prompt for clarification if needed:
 - “Which subject or domain should this plan focus on?”
 - “Would you like the system to consider recent assessment scores?”
- The user responds conversationally, providing inputs like:
 - Student name or identifier
 - Subject focus
 - Learning objective or weakness area

- Timeline for improvement
- Preferred learning style (if known)
- Once enough context is gathered, the system:
 - Validates the student's data and academic records.
 - Calls the AI model tailored for personalisation and curriculum alignment.
 - Generates a structured plan containing:
 - Weekly learning goals
 - Content recommendations (videos, worksheets, readings – internal only)
 - Activity types (quizzes, reflections, group tasks)
 - Assessment checkpoints
 - Suggested support strategies.
- The system displays the full pathway in a structured format within the conversation and provides action buttons:
 - Edit Plan (via follow-up conversation)
 - Save to Student Profile
 - Export Plan (PDF)

e. Field Validations

- Student Identifier: Must match an enrolled student record.
- Subject Area: Must belong to the predefined subject list.
- Learning Objective: Cannot be blank; should be tied to curriculum structure.
- Timeline: Must be numeric (e.g., weeks) and realistic (1–52).
- Learning Style: If selected, must match predefined options or teacher-provided input.

f. Business Validation

- The system must not allow generating a pathway if the student has no academic data uploaded.
- Learning objectives and goals must align with curriculum-defined milestones.
- AI content suggestions must only draw from pre-vetted internal knowledgebase.
- Each saved plan must be assigned a unique version ID and linked to the student's profile.
- Only authorized educators may generate and access personalized pathways.
- If the entered student identifier doesn't exist, system replies:
 - "I couldn't find a student by that name. Please check the name or use the student ID."
- If no data is found for the student, system replies:
 - "This student doesn't have any recent performance data. Please ~~upload or~~ sync their academic records first."
- If a teacher says: "Plan something for a weak student," the system will prompt for clarification on ~~subject~~, name, and objective.
- If the student already has a plan for the same subject and time frame, the system warns the user and asks whether to update, merge, or create new.
- If students try to regenerate or modify their own learning plan, access is denied with a friendly notice:
 - "Learning plans can only be modified by your teacher."

7.4 Collaboration & Resource Sharing for Educators

a. Overview

This module allows teachers and department heads to collaborate on curriculum design, co-create lesson plans, and share content within or across schools. It includes features such as content sharing, version control, and visibility settings to enhance peer collaboration.

b. Preconditions

- Users are authenticated and part of the same organization or collaboration group.
- Repository content exists (uploaded or AI-generated)
- Sharing permissions are defined by the Admin or School Leader

c. Post Conditions

- Shared content is accessible to collaborators.
- All changes are tracked via versioning.
- Comment history and modification logs are retained.

d. Normal Flows

- Teacher opens a content item (lesson plan, quiz, worksheet)
- Clicks “Share”
- A dialog box prompts for:
 - Recipient(s) (via email, role, or team)
 - Permissions: View / Comment / Edit
 - Optional message or context
- Recipient is notified via platform alert or email.
- Shared content appears in folder with a shared icon to indicate the shared content.

e. Field Validations

- Only content with a version ID can be shared.

f. Business Validation

- Noone heads can share content outside the organization.
- Users with Edit access can generate new versions but not delete base content.
- Shared content must maintain original author metadata.
- User leaves organization → access revoked automatically.
- Broken links to shared content → system notifies uploader for re-attachment.

Commented [SA11]: we are not permitting collaboration within the platform, head of departments will be able to view certain activities of their reportees. teachers can download the materials offline and share, they wont be able to via the system

7.5 Classroom Scheduling & Resource Allocation

a. Overview

This module allows teachers and school leaders to query and review student and class performance metrics conversationally. The system analyses data from assessments,

assignments, attendance, and engagement to surface trends, identify ~~atrisk~~at-risk learners, and recommend interventions all within the chat interface.

b. Preconditions

- The user must be a Teacher, Department Head, or School Leadership persona with analytics permissions.
- Student performance data (grades, quiz results, attendance records) must be ingested from SIS or uploaded manually.
- At least one assessment or engagement dataset exists for the period under review.

c. Post Conditions

- The requested analytics are rendered in conversation as summary cards, charts, or narrative insights.
- Users can drill into details via ~~followup~~follow-up prompts (e.g., "Show me the top 5 students by attendance").
- Intervention suggestions are queued for review or assignment to students.

d. Normal Flows

- To Initiate Query User types or selects a quick prompt, for example:
 - "Show me attendance trends for Grade 8 this term."
 - "Which students are below 60% in Math quizzes?"
- System Confirms Filters (if incomplete)
 - If necessary, details are missing, the system asks ~~followups~~follow-ups:
 - "Which date range would you like?"
 - "This term or custom dates?"
 - "Do you want to see individual student data or class averages?"
- The system pulls relevant records, computes metrics (averages, pass/fail rates, engagement scores), and detects outliers.
- The system replies in chat with a structured response:
 - Summary Card: "Grade 8 average quiz score: 72% (↑5% from last term)."
 - Bar Chart Link: Shows distribution of quiz scores.
 - Narrative Insight: "3 students are below the 60% threshold; consider a remediation session."
- User may ask ~~followups~~follow-ups:
 - "List those 3 students by name."
 - "Recommend interventions for them."
- The system returns detailed lists or intervention strategies accordingly.
- The user clicks quick actions:
 - Save Report
 - Export PDF
 - Share

e. Field Validations

- Date Range: Must be valid school calendar dates; start ≤ end.



- Grade/Class Selection: Must match existing classes in SIS.
- Metric Type: Must be one of predefined options (attendance, average grade, engagement).

f. Business Validation

- Only authorized roles can view individual student data and class metrics.
- Insights must reference a minimum data threshold (e.g., at least three data points) to avoid misleading trends.
- Intervention recommendations must align with teacher-configured support strategies.
- If no Data Found:
 - “No quiz records found for Grade 9 this week. Try a different date range.”
- If insufficient Data Points:
 - “Only one attendance record is available; trend analysis requires at least three entries.”

7.587.6 Content Recommendation Engine

a. Overview

This module allows teachers to receive AI-powered recommendations for instructional materials, learning activities, videos, and formative assessments based on student profiles, curriculum alignment, and classroom context. All interactions occur via conversational prompts, with follow-up options to refine, accept, or save the recommendations.

b. Preconditions

- The user must be a Teacher or Educator role.
- Students must have baseline profile data.
- Curriculum mappings must be pre-ingested or defined by the teacher during conversation.
- Knowledgebase with tagged, pre-approved educational content must be available.

c. Post Conditions

- The system recommends context-relevant content (videos, PDFs, quizzes, worksheets) for the teacher to preview, assign, or save.
- Recommendations are saved to the teacher’s personal or shared repository for future use.
- Metadata tags (grade, subject, topic) are applied to saved resources automatically.

d. Normal Flows

- User initiates request
 - Teacher types a prompt such as:
 - “Recommend activities for Grade 6 students struggling with fractions.”
 - “I need some engaging science videos for tomorrow’s lesson on ecosystems.”
- The system clarifies context (if needed)
 - If key details are missing, the system follows up:
 - “What is the learning objective you’d like to target?”
 - “Do you want printable material or interactive content?”

- The system filters from the school's internal content repository, evaluating content suitability based on:
 - Curriculum standards
 - Student learning needs
 - Teacher preferences (e.g., visual, auditory content types)
- The system returns a structured conversational output:
 - Option 1: [PDF Worksheet] Fractions: Pizza Game
 - Option 2: [YouTube Video] Fractions Explained (Grade 6, 8 min)
 - Option 3: [Quiz] Intro to Proper & Improper Fractions
- Each result has buttons: Preview, Assign, Save, Refine
- The teacher can take actions like:
 - The teacher can ask:
 - "Show me only printable worksheets."
 - "Add Option 2 to my lesson plan for Tuesday."
 - The system executes accordingly and confirms actions.
- Recommended content can be tagged and stored in the teacher's content library with metadata such as:
 - Subject: Math
 - Topic: Fractions
 - Grade: 6
 - Assigned Date: [optional]

Commented [SA12]: what would assign and refine mean?

Commented [SA13]: important to clarify that this wont be agentic, i.e., it will generate the text but not integrate anywhere to show info

e. Field Validations

- Topic/Subject: Must align with curriculum taxonomy or predefined subjects.
- Grade Level: Must be within range.
- Content Format Preference: Must be a supported type.
- Learning Objective (optional): Should be text-based and valid for search context.

f. Business Validation

- Only pre-approved and vetted content from the knowledgebase will be suggested.
- Recommendations must be tied to either curriculum standards or user-provided learning objectives.
- Content must align with the student's assigned grade unless overridden intentionally.

7.597.7 Assessment & Evaluation Tools

a. Overview

This module enables teachers to create AI-generated formative and summative assessments such as quizzes, tests, and assignments. It also includes automated grading, rubric-based evaluation, and performance tracking.

b. Preconditions

- User has Teacher or Department Head role.
- Lesson plan or topic is selected.

- Assessment template or structure is chosen (optional)

c. Post Conditions

- A new assessment is generated and saved.
- Grading schema is defined (auto/manual)
- Assessment can be assigned to students or exported.

d. Normal Flows

- User clicks **“Create Assessment”**.
- System prompts:
 - Grade and subject.
 - Topic/sub-topic
 - Assessment type (Quiz, Assignment, Test)
 - Question type preferences (MCQ, Fill in the blanks, Short Answer, Essay)
 - Difficulty level
- System generates:
 - A set of 5–20 questions (based on config)
 - Suggested answer key or rubric.
 - Estimated completion time.
- User reviews and:
 - Edits questions or answers
 - Sets total marks and pass criteria.
 - Assigns to class or saves to repository.
- If assigned, system:
 - Notifies students.
 - Opens submission window.
 - Collects responses.
- Auto-grading occurs for supported question types.
- Teacher can override grades or give manual feedback.

e. Field Validations

- Maximum 50 questions per assessment
- All questions must have a title and correct answer (if auto-graded)
- Submission dates must fall within term calendar.

f. Business Validation

- Students can only attempt once unless configured otherwise.
- Assessment must be linked to at least one curriculum objective.
- Auto-grading not allowed for subjective (essay) questions.
- AI generates duplicate questions → system auto-filters.
- Late submission → flagged but allowed with comment.
- Internet disruption during test → allow draft autosave (future phase)

7.607.8 Lesson Plan Modification & Adaptation

a. Overview

This module allows teachers to modify and adapt existing lesson plans based on real-time classroom needs, changing curriculum requirements, or student feedback. It includes AI suggestions for adaptation and the ability to version-control modified plans.

b. Preconditions

- User has Teacher or Department Head role.
- A lesson plan exists in the system (either AI-generated or teacher-created)
- The user has edit access to the lesson plan.

c. Post Conditions

- The lesson plan is updated and saved as a new version.
- Modification history is tracked.
- Adapted versions can be reused or exported.

d. Normal Flows

- User opens an existing lesson plan.
- Clicks on “Modify Plan”
- System offers options:
 - Add/remove topics or subtopics.
 - Adjust timing or pacing.
 - Replace or add new activities/resources.
 - Change learning objectives.
- Option to “Get AI Suggestions” for the selected changes.
- User reviews and finalizes edits.
- System prompts:
 - Save as new version or overwrite.
 - Add change notes (reason for adaptation)
- Updated lesson plan is saved.
- Prior versions remain accessible under “Version History”.

e. Field Validations

- Modified plan must retain at least one core learning objective.
- Version note required when saving as new version.
- ~~Change summary limited to 500 characters.~~
- Only plan owner or department head can modify lesson plans.
- Curriculum alignment must be retained (warn on misalignment)
- Modifications do not affect already assigned student pathways unless re-synced.
- Plan modified mid-week → prompt to notify students if already assigned.
- Large structural change → system suggests to create a new plan instead.

- Network loss during modification → autosave to local cache (future enhancement)

7.617.9 Audit & Version Control for Curriculum Changes

a. Overview

This module tracks changes made to the curriculum repository, including uploads, edits, deletions, and sharing of content. It maintains a version history for each file or plan and enables rollback or comparison between versions. Designed to support institutional governance and content quality control.

b. Preconditions

- User has Teacher, Department Head, or Admin role.
- Content exists in the curriculum repository.
- Edits or modifications are performed within the system interface.

c. Post Conditions

- Audit logs are updated with change activity.
- Version control is activated for modified content.
- Rollback options are made available where applicable.

d. Normal Flows

- User uploads or edits a content item (e.g., lesson plan, worksheet)
- System logs:
 - User ID
 - Timestamp
 - Change description.
 - Version number
- User can view “**Version History**” from the content’s detail screen.
- From Version History:
 - View changes line-by-line (diff view)
 - Download previous version.
 - Restore older version (admin or owner only)
- Admin dashboard provides:
 - Full activity log
 - Filters by date/user/content type
 - Export option for audit reports.

e. Field Validations

- Change description required when editing content.
- Only .pdf, .docx, .pptx, .mp4, .jpeg/.png accepted for upload.
- Version naming must follow system-assigned format (e.g., v1.2, v2.0)

f. Business Validation

- Users cannot delete version history.
- Admins can lock critical content (e.g., institution-wide templates)
- System retains up to 20 versions per content item by default.
- Two users edit simultaneously → prompt to resolve merge conflict.
- Corrupted version uploaded → system flags and retains prior version.
- Unauthorized access to logs → system denies with audit entry.

8. Functional Requirements – Use Case 3: Student Learning Assistant

8.1 Student Learning Assistant

a. Overview

This module allows students to engage with a conversational AI assistant to ask learning-related questions, clarify concepts, get guidance on study topics, and interact with lesson materials. It operates within a predefined scope configured by the teacher (subject, topic, persona) and is designed for simplicity, contextual relevance, and age-appropriate responses.

b. Preconditions

- Student is logged into the system.
- The associated teacher has activated the Learning Assistant for that subject/class.
- The student has valid class/subject context assigned.
- Safeguarding and moderation filters are active.

c. Post Conditions

- Student receives a helpful AI-generated response to their learning query.
- The interaction is logged in the audit trail.
- Recommendations may be queued for further review or action.

d. Normal Flows

- Student enters the Learning Assistant interface.
- System displays:
 - Context information (e.g., “You are learning Grade 6 Math – Fractions”)
 - Teacher-configured AI persona (e.g., “Visual Learning Coach”)
- Student types or speaks a query (e.g., “I don’t understand how to convert fractions”)
- System performs:
 - Query validation and classification.
 - Context scoping (based on subject, topic, grade)
 - RAG or in-context retrieval from lesson notes, uploaded materials.
- System returns:
 - A structured explanation or guided steps
 - Visuals/examples if supported in the context.
- Student can continue with follow-up queries within the session.
- All prompts and responses are saved in Student Session History

e. Field Validations

- Maximum input length: 500 characters
- Speech input is converted to text with confirmation.
- Output Language: User-enabled, English and Arabic

f. Business Validation

- Responses must not reveal any teacher-configured restricted content.
- Assistant cannot provide direct solutions to assessments unless allowed.
- Age-appropriateness filters must always be active.
- Out-of-scope query → system replies: "I'm only trained to help with your current lessons. Please ask your teacher."
- Ambiguous query → system asks clarifying question.
- Prolonged inactivity → session auto-closes after 10 minutes

8.2 Student Query Input & Recommendations

a. Overview

This module handles the intake of student learning queries and delivers structured recommendations such as follow-up materials, exercises, or supplementary topics based on their questions and academic progress.

b. Preconditions

- Student Learning Assistant module is active.
- Student has interacted with the system within an active session.
- Recommendation engine is enabled and configured by the teacher.

c. Post Conditions

- Student receives 1–3 tailored study suggestions related to their question.
- Suggestions are saved under the session for future reference.
- The system logs which recommendations were clicked or ignored.

d. Normal Flows

- Student submits a query.
- The system analyses:
 - The type of query (conceptual, procedural, revision, etc.)
 - The subject/topic configuration
 - Past student activity and history
- System fetches or generates recommendation cards:
 - "Watch this 2-minute video explaining fractions visually".
 - "Try this worksheet on mixed number conversions".
 - "Revise with this quiz from last week's lesson".
- Cards are shown as clickable suggestions.
- Student can:
 - Click to open the content.
 - Mark as helpful/not helpful
 - Ask a follow-up question.
- Teacher can later view which recommendations were used.

e. Field Validations

- Max 3 recommendations per query

- All content must be pre-approved by the teacher or content repository.
- Recommendation metadata must include subject, content type, duration.

f. Business Validation

- No recommendation should conflict with the current curriculum pacing.
- Personalized recommendations must remain within assigned learning level.
- Content flagged by a teacher must be auto removed from recommendation engine.
- No relevant recommendation found → system prompts user to ask differently.
- Repeatedly ignored recommendations → system adjusts suggestion logic.

8.3 Homework & Assignment Support

a. Overview

This module enables students to seek guidance on homework or assignments through the conversational interface. The system helps clarify concepts and guide students toward solutions without providing direct answers unless explicitly allowed. Teachers configure the scope of assistance.

b. Preconditions

- Student is authenticated and within a valid session.
- The teacher has enabled Homework Support for the subject.
- The assignment is within the configured timeframe for AI assistance.
- Moderation filters are active.

c. Post Conditions

- Student receives conceptual guidance and step-by-step scaffolding.
- Interaction is logged and visible to the teacher.
- Student progress and engagement with the assignment is tracked.

d. Normal Flows

- Student initiates a prompt: e.g., “I’m stuck on question 3 from today’s assignment.”
- System checks:
 - Whether Homework Support is enabled for this subject/task
 - If the assignment is within the active support window
- System responds with:
 - Clarification prompts (e.g., “Can you tell me what’s confusing?”)
 - Hints or examples that relate to the concept (not direct answers)
- Student may upload an image or file of the assignment.
- System parses the content and guides based on the concept.
- Teacher can view the chat log and mark inappropriate usage if needed.

e. Field Validations

- Supported formats: PDF, JPG, PNG for assignment uploads.

- Maximum file size: 10MB
- Speech-to-text input must confirm accuracy before processing.

f. Business Validation

- AI must not output direct answers unless the teacher has explicitly enabled solution mode.
- AI cannot support exams or final assessments.
- All assistance must be curriculum-aligned and reference teacher-approved materials.
- Student uploads unrelated content → system replies with guidance and request for clarification.
- Repeated queries for same question → flag to teacher for manual intervention
- Assignment not yet activated → system replies: "This assignment is not available for AI guidance yet."

8.4 Learning Persona & Goal Selector

a. Overview

This module allows teachers to configure learning personas and set academic or behavioural goals for individual students or groups. The assistant tailors responses based on the selected persona (e.g., visual learner, slow-paced learner) and aligns support to these goals throughout the student interaction.

b. Preconditions

- Teacher is logged in with configuration permissions.
- Students are enrolled and assigned to subjects.
- At least one Learning Persona template exists in the system.

c. Post Conditions

- The student's interaction context is updated with selected persona and goals.
- The AI assistant responds with tone, pace, and examples aligned to that persona.
- Goal tracking is activated in background.

d. Normal Flows

- Teacher opens **Persona & Goal Setup**
- Teacher selects a student or group.
- System displays:
 - Available persona templates (e.g., "Hands-on Learner", "Audio-focused Learner")
 - Goal types (e.g., "Improve math confidence", "Complete 3 exercises per week")
- Teacher selects or customizes:
 - Persona parameters (e.g., response length, tone, visual support)
 - Weekly/monthly learning goals
- Teacher confirms and saves.
- System:
 - Stores persona config with student profile

- Aligns AI responses to the active persona during conversations.

e. Field Validations

- Persona name must be unique.
- Each persona must define at least 3 behavioural traits (e.g., prefers examples, simplified language, asks follow-up questions)
- Goals must include measurable metric or timeline.

f. Business Validation

- Only teachers (not students) can assign or change personas.
- Personas must be curriculum-aligned.
- No persona can lower safeguarding standards or override moderation.
- Multiple personas selected → system defaults to most recently configured one.
- Unrecognized persona config → system reverts to default assistant behaviour.
- Goal expiry without achievement → system logs and notifies teacher.

8.5 Upload Classroom Materials for Context

a. Overview

This module allows students to upload classroom materials (e.g., class notes, slides, worksheets) to be used by the conversational AI assistant for more context-aware and personalized responses. All uploaded content is visible to the associated teacher for moderation and control over what the assistant uses.

b. Preconditions

- Student is logged in and assigned to an active subject or class.
- Uploads are permitted by teacher for that subject.
- File upload size and type restrictions are enforced.
- Teacher moderation tools are active.

c. Post Conditions

- Uploaded materials are processed and indexed for in-context retrieval (RAG)
- AI assistant uses validated content for query responses.
- Upload is logged and linked to the student profile and conversation session.

d. Normal Flows

- Student enters the assistant and clicks **"Upload Notes or Slides"**.
- System prompts for:
 - File upload (PDF, PNG, DOCX)
 - Optional tag or title (e.g., "Math - Fractions Notes")
- Student uploads file
- System validates file and performs:
 - Virus scan

- OCR/text extraction
 - Metadata generation
- System routes file for **Teacher Moderation Queue**
- Teacher reviews and marks content as:
 - Approved for AI context.
 - Rejected or flagged.
- Once approved, content is ingested for RAG use with prompts scoped to that class/topic.

e. Field Validations

- Supported formats: PDF, DOCX, PPTX, JPG, PNG
- Max file size: 10MB per file
- Max uploads per student per day: 5 (configurable)

f. Business Validation

- Unmoderated content cannot be used in AI prompt responses.
- Content is only accessible for that student's class/subject context.
- System must maintain traceability: who uploaded, when, and for what class
- Duplicate files → system alerts the student before re-uploading.
- Unreadable content → system rejects and notifies user.
- No content approval in 48 hours → soft-expiry, reminder sent to teacher.

8.6 Progress Tracker & Feedback Summary

a. Overview

This module automatically tracks student engagement, learning patterns, and assistant usage to generate weekly feedback summaries. Summaries highlight topic mastery, struggling areas, and personalized next steps. Teachers can view dashboards; students get conversational feedback summaries.

b. Preconditions

- Student has used the assistant during the week.
- Learning goals and personas are assigned.
- The system's analytics service is active and accessible.

c. Post Conditions

- Student receives a conversational summary with insights.
- Teacher gets access to student-level and class-level analytics.
- Summary and trends are stored for longitudinal tracking.

d. Normal Flows

- Weekly (or teacher-triggered), system processes student activity:
 - Queries asked.
 - Materials used.

- Recommendations clicked.
 - Goals achieved or missed.
- System generates:
 - Summary narrative for student (e.g., “You’ve made great progress in decimals!”)
 - Highlighted focus areas (e.g., “Review multiplication of fractions”)
 - Teacher dashboard metrics (engagement %, time-on-task)
- Student sees summary in chat with options to:
 - Review suggested materials.
 - Ask follow-up questions.
- Teacher sees:
 - Graphical reports
 - Struggling students by topic
 - Recommendation effectiveness

e. Field Validations

- One summary per week per student
- Teacher dashboard must support filters: class, subject, student, timeframe.
- Progress thresholds for “improving”, “stable”, “needs support” must be defined.

f. Business Validation

- Data must not be shown to any role outside teacher/student unless explicitly shared.
- No AI-generated summary may include personally sensitive observations.
- System must not perform automated grading—only interpret assistant usage.
- No assistant activity during week → system skips summary.
- Low engagement → system prompts teacher with suggestion to intervene.

8.7 Student Guardrails & Prompt Moderation

a. Overview

This module ensures student safety, appropriate use, and educational relevance by applying real-time moderation, filtering, and prompt-level validations. It enforces content boundaries, prevents misuse, and ensures all AI responses remain age-appropriate, contextual, and aligned with school policies.

b. Preconditions

- Student session is active and authenticated.
- The assistant is operating under a teacher-assigned configuration.
- School-wide moderation settings have been defined.

c. Post Conditions

- Unsafe or inappropriate prompts are blocked or redirected.
- Moderation logs are generated and stored for audit.
- Teacher may be alerted for flagged prompts based on rules.

d. Normal Flows

- Student submits a prompt (typed or voice)
- System performs:
 - Real-time scanning of content using predefined filters.
 - Pattern recognition for inappropriate or sensitive language.
 - Validation against curriculum scope and behavioural models
- Based on result:
 - Blocked Prompt → System returns: *"This question isn't allowed. Please ask something related to your lesson."*
 - Sensitive Prompt → System allows but flags to teacher for review.
 - Safe Prompt → Routed to LLM for response.
- Moderation log is created, including:
 - Prompt content
 - Action taken.
 - Timestamp and student metadata.

e. Field Validations

- Max prompt length: 500 characters
- Keywords, phrases, and intent filters configured per school.
- Voice input is transcribed and passed through same moderation engine.

f. Business Validation

- Moderation cannot be bypassed by student-level configurations.
- All blocked prompts must be visible in teacher moderation reports.
- No prompt should reveal information about other students or system internals.
- False positive (e.g., "bomb calorimeter" in chemistry) → flagged for whitelist review.
- Repeat violations from student → optional escalation workflow.
- Multilingual prompts → translation required before moderation (future roadmap)

8.8 Teacher Control Console

a. Overview

This module enables teachers to configure, monitor, and control AI behaviour for their respective classes. It includes persona assignment, content approvals, moderation logs, student usage reports, and the ability to trigger system overrides or pauses in real time.

b. Preconditions

- Teacher is logged in with proper permissions.
- At least one subject/class is assigned to the teacher profile.
- Students are enrolled and active under teacher's purview.

c. Post Conditions

- Class-level AI assistant configurations are saved.

- Teacher can monitor and act on student-AI interactions.
- Overrides and updates apply immediately to student sessions.

d. Normal Flows

- Teacher logs into the system and accesses **Control Console**
- Teacher views:
 - Current configuration status for each class
 - Assigned personas and learning goals.
 - Moderation alerts from recent student prompts
- Teacher can:
 - Approve or reject uploaded materials.
 - Change student personas or deactivate assistant per student.
 - Trigger emergency pause (e.g., "Stop AI Assistant for Class 7A")
 - View session logs and usage metrics per student
 - Preload content and predefined prompts.
- Changes made are versioned and take effect immediately for the selected class/students.

e. Field Validations

- Only assigned classes visible to teacher
- All configuration changes are timestamped and auditable.
- Emergency pause requires confirmation dialog.

f. Business Validation

- Teacher cannot configure LLM behaviour outside predefined system rules (e.g., can't disable moderation)
- All student logs accessed must be tied to the teacher's assigned cohort.
- Uploaded content must be reviewed before becoming context for student prompts.
- Teacher attempts to configure assistant for unassigned class → system blocks access.
- Simultaneous edits → system resolves using latest save with versioning log.
- Content not reviewed in time → student uploads expire and are archived.

9. Functional Requirements – Use Case 4: Admissions & Marketing

9.1 Admissions Process Assistant

a. Overview

This module provides conversational support for school staff to manage and streamline the admissions process. It handles queries related to application status, required documents, applicant eligibility, interview scheduling, and more—through natural language interactions.

b. Preconditions

- Admissions staff is authenticated and assigned the appropriate role.
- Admission season is open or configured in the system.
- Applicant data is ingested into the system (either manually or via SIS integration)

c. Post Conditions

- Staff receives accurate responses or actions triggered related to the admission workflow.
- System captures conversation history for each applicant session.
- System updates application status if permitted by user role.

d. Normal Flows

- Staff enters a query:
 - “Show me pending applications for Grade 7”.
- System retrieves filtered applicant list.
- ~~Staff may continue:~~
 - “What documents are missing for Fatima’s application?”
- System checks document tracker and returns:
 - “Proof of residency is pending.”
- Staff may request:
 - “Send reminder email to parent” → system drafts and displays message.

e. Field Validations

- Grade levels must be valid and mapped to current school structure.
- Interview slots must not overlap; calendar conflict check required.
- Applicant IDs must exist and be active.

f. Business Validation

- Only authorized roles can trigger updates (e.g., scheduling, status change)
- No application status changes can occur after cutoff date unless override is enabled.
- Staff cannot view applications outside their assigned academic division.
- Duplicate application queries → system flags and provides potential matches.
- Incomplete records → assistant highlights missing fields before proceeding.
- Scheduling beyond school hours → system prevents and suggests alternatives.

9.2 Applicant Profile Evaluation

a. Overview

This module allows the AI system to parse, summarise, and evaluate applicant profiles—including academic records, entrance tests, and supporting documents—to highlight strengths, concerns, and missing information.

b. Preconditions

- Applicant profile is uploaded and processed (PDF, DOCX, image-based OCR, etc.)
- Staff is logged in and has access to the applicant's assigned grade level.
- Evaluation settings (rubrics or guidelines) are defined.

c. Post Conditions

- A narrative-style profile summary is generated for each applicant.
- AI flags missing or concerning items (e.g., low grades, missing forms)
- Summary is stored with timestamp for audit and available to reviewers.

d. Normal Flows

- Staff selects or asks:
 - “Evaluate Anaya Kapoor’s application”
- System checks:
 - Profile completeness
 - Uploaded academic records.
 - Entry test results (if applicable)
- System generates output like:
 - “Anaya has consistent academic performance, with strength in science. Entry test scored 88%. Missing reference letter. Overall profile aligned with Grade 9 admissions expectations.”
- Option to download or save evaluation summary is provided.
- Option to mark applicant as flagged or recommended for interview.

e. Field Validations

- File upload types: PDF, DOCX, image formats (OCR supported)
- Max file size: 10MB
- Entry test score field must be numerical between 0–100.

f. Business Validation

- AI should not generate conclusive “admit/reject” decisions—only recommendations.
- All AI-generated insights must include source markers (e.g., “Based on GPA in last 3 years”)
- Staff cannot view applicants outside their assigned scope.
- Unreadable scanned document → system flags and requests re-upload
- Multiple uploads for same document type → system prompts user to select preferred one.

- Inconsistent data across fields → system highlights and marks for manual review

9.3 Admissions Insights Dashboard

a. Overview

This module provides conversational and visual access to real-time admissions data, including application volumes, stage-wise conversion, demographic trends, and funnel performance. The assistant delivers summarized insights, while the dashboard UI allows staff to filter and explore data more granularly.

b. Preconditions

- Admissions data is being collected and structured (via manual entry or SIS integration)
- User is authenticated and assigned to an Admissions or Leadership role.
- Data aggregation services are running and up to date.

c. Post Conditions

- User receives summarized and visual insights based on selected filters.
- Reports are generated or downloadable for leadership presentations.
- Assistant can use this data context to suggest actions (e.g., follow-up with inactive leads)

d. Normal Flows

- Staff enters conversational query:
 - “Show me this month’s application conversion rate”.
- System processes and returns insight:
 - “28.4% of 134 applications have reached interview stage”.
- User can ask:
 - “Which grades have the highest dropout rate?”
- System presents bar chart + narrative insight.
- Optional: Open detailed dashboard view with filters:
 - Date Range
 - Grade Level
 - Source Channel (Website, Agent, Social Media)
 - Application Status
- Option to download as PDF or CSV report.
- AI can suggest insights:
 - “Grade 4 applications are down 20% compared to last year. Consider campaign boost.”

e. Field Validations

- Date fields must be valid ranges.
- Filter selections must map to configured metadata (grades, channels, statuses)

f. Business Validation

- Only summary-level data shown unless user has advanced access.
- No personal student/parent data visible in dashboard
- Insights must be grounded in data—no speculative trends without basis.
- Missing data for selected time window → system notifies: “No data found for April 2024”.
- Overlapping filters → system auto-adjusts or alerts user.
- High latency in real-time sync → cached view served with timestamp notice.

9.4 Enquiry Management & Follow Up Generator

a. Overview

This module allows marketing personnel to generate AI-driven visual and textual content for promotional campaigns across platforms like Instagram, LinkedIn, Facebook, etc. The interaction occurs in a multi-turn conversation: the system first asks the platform, then the theme or content prompt, and finally generates a preview image. Users can optionally edit the output within a built-in editor before downloading or using it.

b. Preconditions

- Marketing user is authenticated with the appropriate role.
- System has access to creative generation ~~LLM/VLM (Vision Language Model)~~ services locally ~~or via offline hosted APIs~~.
- Content prompt, platform selection, and school brand configuration are in place.

c. Post Conditions

- A generated campaign asset (image + text) is produced for the selected platform.
- User can either regenerate, edit, or download/share the asset.
- Edited versions are stored and tracked in user history.

Commented [SA15]: image will not be produced, it will be selected from the school's existing image/assets directory

d. Normal Flows

- User enters prompt:
 - “Create a post to promote our Grade 6 admissions”.
- System replies:
 - “Which platform is this post for? (Instagram, LinkedIn, Facebook, Other)”
- User selects “Instagram”.
- System asks:
 - “Any specific tone, keywords, or theme you'd like to include?” (Optional)
- User replies:
 - “Highlight STEM, fun learning, and small class sizes”.
- System generates a visual + caption (Instagram-style preview card)
- Below the preview, buttons appear:
 - Regenerate
 - Edit
- User clicks **Edit** → Editor opens with the following editable options:
 - Text Content

- Font Family
 - Font Colour
 - Font Size
 - Bold / Italic / Underline
 - Add Numbered or Bulleted List
- User finalizes edits and clicks **Save**.
- Image is saved as a new version in **Campaign Library** with metadata (platform, date, creator)
- Users with admin access should be able to see a Branding Tab in the Configuration module. This should allow the user to configure:
 - Uploading brand logos (JPG, PNG)
 - Defining brand colour palette (primary, secondary, tertiary HEX Codes)
 - Setting default font family and text alignment. (Font family cannot be changed by downloading fonts in the system)

e. Field Validations

- Platform choice must match one of the supported: Instagram, LinkedIn, Facebook, Twitter
- Prompt must be ≤ 500 characters.
- Max image resolution: 1080x1080 (Instagram), 1200x627 (LinkedIn), etc.
- Edited text length must stay within social media platform limits.

f. Business Validation

- Only approved brand logos, colours, and font styles can be used (configured in brand kit)
- System prevents uploading of copyrighted or offensive materials.
- Regeneration after editing resets prior changes unless saved.
- User selects unsupported platform → system returns: "Currently we support Instagram, LinkedIn, and Facebook."
- System fails to generate due to GPU resource unavailability → fallback message shown.
- Image content violates moderation rules → system warns and blocks asset.

9.419.5 Campaign Performance Insights

a. Overview

This module provides insights into the effectiveness of marketing campaigns. It uses integrated data or manual entry to track views, engagement, conversions, and churn. Results are accessible through conversational prompts and/or graphical dashboards.

b. Preconditions

- Performance data is either manually uploaded or ingested from connected campaign systems.
- Campaign metadata (platform, time, audience, theme) is logged during generation or upload.
- User is authenticated with Marketing or Admin role.

c. Post Conditions

- User receives visual and narrative feedback on campaign success metrics.
- Recommendations for improvement are generated.
- Reports are stored for future comparison.

d. Normal Flows

- User asks:
 - “How did our Instagram campaign for Grade 8 admissions perform last week?”
- System returns summary:
 - “Views: 12.3K | Clicks: 934 | Applications Started: 62 | CTR: 7.6%”.
 - “Engagement was 22% lower than average — consider simplifying CTA.”
- User clicks View Graph → Bar chart + trendline appears.
- Optional filters:
 - Date range
 - Platform
 - Campaign type (admission, branding, event)
- Download or export report to PDF.
- AI may suggest next step:
 - “Would you like to generate a follow-up campaign?”

e. Field Validations

- Campaign IDs must match existing records.
- Metrics must be numerical and within reasonable range.
- Custom KPIs (e.g., application-to-click ratio) must have valid data sources.

f. Business Validation

- Only users with Marketing Analytics access can view raw metrics.
- Historical campaigns older than 12 months archived unless retained by admin.
- No individual user data shown—only aggregate statistics.
- No tracking data found → System shows: “We couldn't find performance data for this campaign.”
- Metric anomalies (e.g., 0% CTR) → AI flags and prompts user for manual validation.
- Inconsistent campaign names → system prompts to merge or clarify.

10. Non-Functional Requirements

Non-Functional Requirements define the quality attributes, performance benchmarks, and system constraints that a software application must adhere to. They ensure the system is usable, reliable, scalable, and secure, complementing the functional requirements.

a. Performance

- ~~The system shall respond to user inputs (text or voice) within 2 seconds for standard prompts and within 5 seconds for model-based generation tasks under typical load.~~

- For AI-generated reports or recommendations, ~~initial~~cached content rendering shall occur within 7 seconds, excluding large batch documents.
- All modules should operate fully offline without degradation in performance when disconnected from the internet, provided intranet and local infrastructure is active.
- The system shall maintain conversational continuity with no session freeze during model switching or chaining between use cases.
- Report rendering, audio playback, and chat summarization must operate smoothly on edge hardware provisioned as per baseline configuration ~~(TBD)~~.

b. Scalability

- The system shall be capable of scaling to support different school sizes, from 5 to 500 concurrent users, depending on licensing tier and hardware allocation.
- The architecture must support horizontal scalability via modular deployment of LLMs, enabling the addition of new use case models without re-architecting the core platform.
- Knowledgebases, prompts, and model configurations should be maintained per organization/school.
- System performance must not degrade when multiple AI use cases are queried in succession.
- The offline-first sync architecture shall support deferred sync queues and file-based batch updates for knowledgebase and SIS data across multiple sites.
- Hardware provisioning (GPU, storage) must allow upgrade paths in terms of compute and memory to accommodate increased user base or feature complexity.

c. Security

- The system shall operate in a fully on-premises environment, with no dependency on external cloud services.
- All user data, including chat history, student activity logs, uploaded files, and generated reports, shall be stored locally within the organization's infrastructure with AES-256 encryption at rest.
- User authentication and role-based access control shall be enforced across all modules to prevent unauthorized access, including read/write restrictions per user type.
- Audit trails will be maintained for all user activities, especially for sensitive roles like students, inspectors, and administrators, with timestamps and action types.
- Students will operate under supervised access only, with strict moderation of prompts, and no exposure to public internet or unvetted content.
- The platform will support multi-languagebilingual data protection, ensuring encryption and access control extend to both English and Arabic inputs/outputs.
- Editing rights for AI-generated content will be restricted by role to avoid manipulation of official documents like inspection reports.
- The system shall support admin-triggered user session termination and manual data purging, where required, to comply with internal policies.
- All internal APIs, including between UI, orchestration layer, and ~~NIM~~ models, must use mutual TLS or equivalent secure local communication protocols.
- Access to the Super Admin Panel shall be limited to a predefined set of users ~~with multi-factor authentication (MFA) if hardware permits~~.

d. Usability

- The system shall present a unified conversational interface to all users, reducing the learning curve and maintaining a consistent experience across use cases.
- Users shall be able to interact with the platform via text and voice in both English and Arabic, with language toggling available at the UI level.
- The UI shall provide predefined prompts, quick actions, and suggestions to guide non-technical users through complex workflows.
- For students and teachers, the system will support context-aware interactions, allowing them to continue a previous conversation or resume unfinished tasks.
- The interface will be accessible on modern desktop browsers and optimized for common classroom devices defined during deployment.
- All user-facing flows shall prioritize clarity, readability, and minimal cognitive load, especially in modules like lesson planning, student Q&A, and inspection report analysis.
- Where applicable, the system shall allow users to download, listen to, or regenerate content without leaving the current screen or losing context.
- The admin feature screens will follow a functional dashboard pattern rather than conversational UI, ensuring rapid configuration access.

Commented [SA16]: voice only in English, no Arabic

e. Maintainability

- The platform shall follow a modular architecture, with separate services for each major use case, allowing individual modules to be updated without system-wide downtime.
- ~~Model orchestration logic will be configurable via a Super Admin Panel, allowing AI model selection, fallback messages, and chaining logic to be modified without code redeployments.~~
- The system shall support versioning of AI models and configurations, allowing rollback to earlier states in case of unexpected behaviour post-deployment.
- Logs, error messages, and system diagnostics will be exposed via a System Health Monitoring module, supporting root cause analysis by the implementation team.
- The file-based sync mechanism must include clear logging, retry logic, and conflict resolution strategy to ensure maintainability in partially connected environments.
- All the settings should be easily manageable by non-technical users, with minimal dependency on backend changes.

f. Reliability & Availability

- The system shall be designed for high reliability in an offline-first environment, ensuring full platform functionality without internet connectivity.
- All critical user flows including lesson planning, student interaction, inspection report generation, and admissions support must be operable even during network disruptions within the school LAN.
- The system shall maintain a local cache of recent queries, knowledgebase content, and generated outputs to prevent loss during session interruptions or power fluctuations.
- Core services (AI model containers, orchestration engine, local file-based sync, database) shall be monitored continuously by a health check service, with failure notifications made available to the Super Admin.
- Sync processes with ~~external~~ local, school-provisioned data sources (SIS, internal KB) shall include retry logic, logging, and local backup to prevent data inconsistency due to failed transfers.

- All user data, model configs, and generated content shall be regularly backed up to a designated local storage path that can be exported if required.
- The system shall be capable of operating for a minimum of 12 hours of active use daily under school load, assuming minimum specified hardware is provisioned.

11. System Requirements

11.1 User Interface

- **Component Library:** Use React to build modular, reusable UI components with efficient virtual-DOM rendering.
- **Styling Framework:** Apply Tailwind CSS for utility-first, responsive styling that keeps the CSS bundle lightweight and design tokens centralised.
- **State Management:** Integrate Zustand as the primary global store (minimal bundle, predictable data flow); supplement with React Context for local, low-complexity state.
- **Data Fetch & Caching:** Utilise TanStack Query for declarative server-state management, background revalidation, and offline cache hydration on the appliance.
- **Routing:** Rely on Next.js App Router (file-based routing) to handle navigation across the four core modules: Inspection, Curriculum, Analytics, and Admissions, delivering a seamless single-page application experience.
- **Animation:** Implement Framer Motion to add smooth micro-interactions and transitions.
- **Rich Editing (Marketing Module):** Lazy-load Fabric.js only when the Admissions/Marketing workspace is opened, enabling canvas-based poster and banner design without bloating the base bundle.
- **Accessibility & Internationalisation:** Provide Right-to-Left (RTL) support for Arabic via Tailwind's RTL variant; manage translations with next-intl message catalogues.

11.2 Database

- **Database Management System (DBMS):** Utilise PostgreSQL as the primary relational database management system for structured application data, offering full ACID compliance and advanced indexing support. PostgreSQL shall be deployed in an on-premise, offline configuration to satisfy data residency, privacy, and regulatory offerings.
- **Vector Database Engine:** Incorporate FAISS as a local semantic vector store to handle unstructured data (e.g., content embeddings from PDFs, chat logs, lesson plans). Metadata references and vector index versions shall be cross-linked in PostgreSQL.
- **Data Models:** The database schema shall support the following core entities and relationships:
 - User accounts and roles (admin, teacher, student)
 - Uploaded file metadata and ingestion status (PDFs, DOCX, XLSX, CSV)
 - Conversation histories and query logs
 - Knowledgebase configurations (teacher-level memory, central sources)
 - Inspection source alignment and tagging
 - Vector index metadata and refresh history

- **Data Storage Strategy:** Store raw documents in a local file system or object store, with references in PostgreSQL. Extracted text shall be parsed and stored in FAISS or similar vector DBs for Retrieval-Augmented Generation (RAG) workflows.
- **Scalability & Performance:** Design the schema with optimized foreign keys, partial indexes, and query plans to support:
 - Real-time RAG lookups across multi-modal sources
 - Bulk embedding pipeline support (batch writes to FAISS)
 - Local full-text search using PostgreSQL's tsvector or integrated search extensions
- **Backup & Recovery:** Configure automated daily backups of both PostgreSQL and FAISS indices to an encrypted local directory.
- **Data Retention & Lifecycle Management:** Support configurable retention policies per data type:
 - Unused uploads (PDFs, spreadsheets) purged after 90 days if unlinked
 - Student chat history retained 30–60 days (school-configurable)
 - Knowledgebase versioning retained indefinitely unless manually cleared

11.3 Communication/Security

- **Secure Communication Protocols:** Enforce HTTPS with TLS 1.2 for all service-to-service and browser connections. Internal gRPC/REST traffic between micro-services shall use mutual TLS (mTLS) issued by an on-box Certificate Authority. API endpoints will bind only to localhost or the school LAN subnet.
- **Authentication and Access Control:** Store passwords using strong hashing algorithms. Implement secure session handling with timeouts and protection against session hijacking.
- **Data Encryption and Protection:** Encrypt sensitive data in databases and storage systems. Collect only necessary data and avoid storing sensitive information like payment card details.
- **Network Security:** Deploy firewalls to protect servers from unauthorised access.
- **Incident Response and Logging:** Develop procedures for detecting, responding to, and recovering from security incidents. Maintain detailed logs of system access and user activities for auditing purposes.
- **Compliance with Regulations:** Ensure data handling practices comply with data protection regulations in target regions. Obtain and manage user consent for data processing activities.
- **Third-Party Security:** Securely store and manage API keys and credentials for external services. Evaluate third-party service providers for their security practices.

11.4 System Integration

- **Local Knowledge Base & Vector Store Integration:** Deploy a FAISS vector store on the appliance for embedding persistence and similarity search; maintain cross-references to PostgreSQL for versioning and access control. Integrate an on-device LLM layer (default Qwen-2.5-7B, optional Llama-3-8B) via GGUF-quantised weights to fit ≤ 8 GB VRAM or CPU-only fallback. Couple retrieval with a bge-reranker-base model running in duplex mode. All Retrieval-Augmented Generation (RAG) operations execute entirely offline with no external API traffic. NGC Catalog models will be leveraged wherever applicable to serve for accelerated inference.
- **Document Processing Pipeline:** Orchestrate a micro-service that chains

- File parsers: PyMuPDF (PDF), python-docx (DOCX), pandas (CSV/XLSX)
- Text preprocessing: rule-based cleaner → hierarchical chunking (512 token windows, 20 token overlap) → metadata tagging (author, academic year, KHDA strand)
- Embedding generation: bge-m3 (mixed-modal encoder) batched; auto-switch to GPU if available.
- **RAG Service Contracts:** Expose internal endpoints Embed(), Search(), GenerateAnswer(), and more as applicable; downstream UI calls gateway. Each request carries a signed JWT to enforce RBAC at the service layer.
- **Text-to-Poster (Marketing Module):** System selects an image from media path defined by the admin, edits on-canvas overlays with Fabric.js.
- **Chat Interface Integration:** Route all prompts through an agent orchestration framework (like Langchain): 1 → Retrieve() from FAISS 2 → Rerank() 3 → LLM.generate() with streaming tokens to the Next.js front-end. Support ad-hoc document upload for selected workflows: new file parsed, embedded, and temporarily added to the user's session index for contextual Q&A.
- **Event Logging & Audit Trail:** Emit structured JSON logs (chat_event, file_ingest, vector_op) for logging and auditing purposes.
- **Sync & Portability:** By default the appliance is siloed; however, a manual on-premise export/import CLI (mis-export --all, mis-import <file>) enables school-to-school migration or seeding of a cloud tenant, preserving vector IDs and metadata hashes.

11.5 Constraints and Limitation

a) Constraints

- **Deployment Model:** The solution is designed as an offline appliance deployed on the school's LAN; multi-tenancy cloud is a future option. All core services must run locally without relying on Internet connectivity for now.
- **Hardware Envelope & Scaling:** A tiered approach is planned with a basic PC, mid-level, and high-level servers, scaling VRAM and CPU for larger user pools.
 - Schools wanting > ~10 concurrent users require the mid/high tier
 - Whereas school wanting > 100 will be treated as enterprise.
- **Concurrent User Cap:** The base tier of the appliance must deliver acceptable performance for 5 – 10 simultaneous users; exceeding this may introduce latency unless upgraded hardware is purchased.

b) Storage Quota & Upload Guardrails

- **Total document quota:** soft warning at 45 GB, hard lock at 50 GB until the school deletes data or upgrades.
- **Per-file size:** ingest accepts files up to 150 MB, with compression wherever possible.
- **Language Coverage:** Out-of-the-box support is English, with experimental Standard-Arabic via on-the-fly translation; Arabic queries may yield lower fidelity than English.
- **Network & Bandwidth:** Schools are responsible for ensuring adequate LAN bandwidth; MIS Consultancy will not manage campus networking or bandwidth issues.
- **High Availability:** No cross-site replication or automatic fail-over. Resilience is limited to local service restarts; optional same-LAN PostgreSQL replica if the school provides hardware.
- **Model & Vector Limits:** Appliance ships with models that fit the specified VRAM.

- **Browser Scope:** UI is optimised for modern Chromium-based browsers and Firefox ≥ v115.

11.6 Legal

11.6.1 Data Protection and Privacy

- **Local Data Control:** All data (documents, PDFs, student info, inspection material) will be stored and processed locally within the institution's infrastructure, ensuring full control by the school.
- **No External Data Transfer:** The system does not send or expose any data to external servers or third-party APIs. All LLMs, vector stores, and processing pipelines run offline.
- **Consent & Ownership:** Schools retain ownership of their data. Teachers or admins uploading documents must ensure they have proper consent or authority to use the data with the system.
- **Student Data Protection:** When student-related content (learning support, uploaded homework) is used for AI responses, proper internal policies must be followed to avoid misuse or over-retention.
- **Right to Deletion:** Admins can delete documents, vector indexes, or chat histories at any time. No personal data is retained once deleted.

11.6.2 AI Content and Educational Use

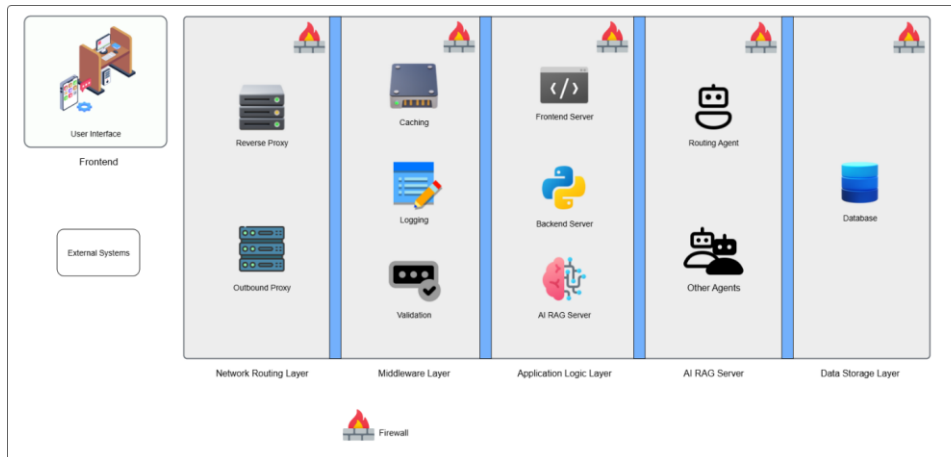
- **Use Limitation:** The chatbot provides suggestions, summaries, and plans using school-provided content. It is not a replacement for professional decision-making or inspections. Final judgments should always be made by qualified staff.
- **Editable Outputs:** Any generated reports (e.g., mock inspection reports or lesson plans) are editable and meant to be reviewed before external use.
- **Content Review Responsibility:** The school or teacher is responsible for validating and approving any AI-generated responses used in documents, presentations, or communication.

11.6.3 Liability Limitation

- **No Liability for Inaccuracies:** The platform provider is not responsible for incorrect, incomplete, or misleading output generated by the LLM. It is the user's responsibility to validate final content.
- **School Responsibility for Data Handling:** The school or institution deploying the system is responsible for proper governance of data (e.g., uploading correct inspection files, not exposing sensitive student info).
- **No Internet/Cloud Dependency:** Since the platform runs fully offline, no legal obligations exist around cross-border data sharing, third-party data processors, or cloud terms of service.

12. System Architecture

12.1 Overview



The system is architected as a secure, high-performance, fully offline AI platform for schools. It follows a modern, containerised N-tier design that keeps every byte of data on campus while allowing future lift-and-shift to a multi-tenant cloud edition. Project workshops confirmed a single-box focus first, with cloud left for phase-two.

- **Privacy-first deployment:** All LLM inference, vector search, and storage run locally; the box never calls external APIs during normal operation.
- **Retrieval-Augmented Generation (RAG) core:** Documents are parsed, chunked, embedded, stored in FAISS, then retrieved → re-ranked → passed to a quantised LLM; chunk size and continuity were highlighted as “the most important part” for answer quality.
- **Modular micro-services:** Containers for auth, ingest, rag, and admin, enabling per-use-case licensing and straightforward upgrades.
- **Tiered hardware path:** Basic, mid, and high SKUs scale CPU/RAM/GPU to support from ~10 to ~100 users, as sketched in the hardware-spec action items.

12.2 AI Subsystems

This is a core component unique to this platform responsible for all intelligence pipelines.

12.2.1 Router & Orchestration Layer

- **Task:** At runtime decide which specialised engine (Inspection, Curriculum, Classroom, Marketing, FAQ) should serve the incoming user turn; attach any necessary system prompts, retrieval hooks and memory slices before forwarding.
- **Model:** Phi-3-mini-4k-instruct (3.8 B params, 4 k context). azure.microsoft.com

- **Rationale:**
 - State-of-the-art few-shot classification accuracy on MMLU-style tasks while remaining small enough to run once **per turn** without becoming the bottleneck.
 - Phi-3 ships with an open, MIT-compatible licence.

12.2.2 Knowledge-Centric Q&A Route

- **Model:** Qwen-2.5-32B-Instruct huggingface.co
- **Core tasks:**
 - Long-form academic answers, citation-centric explanations, Socratic tutoring.
 - Multi-turn follow-ups over 16 k tokens (course material + chat history).
- **Rationale:**
 - Reasoning depth: top-3 on Needle-in-a-Haystack 10-shot; strong GSM-8 k and MATH scores.
 - Context breathing room: 32 k window reduces retrieval misses when the student pastes full articles.
- **Execution pattern:**
 - Retrieve top-20 chunks via BGE-M3 embeddings.
 - Re-rank to top-6.
 - Insert into Qwen-2.5 system prompt as ### Sources: block.

12.2.3 Curriculum & Lesson-Plan Composer

- **Model:** Qwen-2.5-32B-Instruct (same weights, separate endpoint with pedagogy-focused system prompt).
- **Core tasks:**
 - Generates weekly schemes of work, Bloom-aligned objectives, slide outlines and formative quiz items.
 - Accepts JSON spec {age band, syllabus, learning style, prior knowledge}.
- **Rationale:**
 - Maintains pedagogical tone consistency with Q&A route.
 - Reduces ops overhead (same checkpoints, one quantisation format).
- **Additional Safeguards:**
 - Post-generation rubric checker (regex-based) flags missing success-criteria or off-syllabus content.

12.2.4 Analytical Report & Code-Generation Agent

- **Model:** DeepSeek-Coder-33B-Instruct (32 k context, function-calling tuned). huggingface.co
- **Core tasks:**
 - Generates Python/Typescript notebooks, plots, tabular summaries and APA-formatted reports.

- Able to emit JSON + base64 images for the front-end renderer.
- **Rationale:**
 - **Code Reliability:** Beats GPT-4-Turbo on HumanEval by +4.8%
 - **Tool-use readiness:** Native function-call tokens reduce hallucinated filenames.
 - **Large Context:** Hold entire CSV ≈ 8 MB plus instructions in a single pass.

12.2.5 Vision & Inspection Agent

Used when the router [12.2.1](#) detects an image.

- **Model: Llama 3.2-Vision 3B**
 - Meta's 3-billion-parameter multimodal checkpoint (image encoder + Llama 3.2 text decoder) runs comfortably on a single 8-12 GB GPU / Apple-M-series laptop while beating earlier open-source vision models on VQA, captioning and multimodal reasoning benchmarks.
- **Core tasks:**
 - Alt-text generation, simple OCR.
- **Rationale:**
 - **Offline-first:** No cloud calls; weights are Apache-2.0 and can live on the same server as the text LLMs.
 - **Footprint vs. Fidelity Trade-Off:** At ~4 GB GGUF (Q4_K_M) the model leaves head-room for concurrent text agents on the same box without starving VRAM, yet is accurate enough to locate hazards, count objects, and parse signage.
 - **Unified Tokenizer:** Shares Llama-3 token space with text-only agents, so we can feed mixed image-and-text prompts into the broader orchestration layer without extra adapters.

12.2.6 Retrieval Infrastructure (shared by all routes)

- **Embeddings:** BGE-M3 (base, 768 d) supports dense + lexical + ColBERT multi-vector retrieval, and 8 k+ token passages. huggingface.co
- **Re-ranking:** BGE-reranker-base cross-encoder – +9 nDCG@10 on BEIR vs vanilla cosine; multilingual and 8 k context. huggingface.co
- **Vector store:** Open-source FAISS with separate collections per tenant to honour FERPA/GDPR isolation.
- **Flow:**
 - Text chunks (300–500 tokens) are embedded on write.
 - At query time the router chooses whether to call retrieval; if yes, top-k (dense = 24 | lexical = 24 | MV = 12) are fetched.
 - Reranker trims to top-n (n = 6 default, 12 for audits).
 - The chosen generative model receives documents[i].content + documents[i].metadata.source.
- **Rationale:**
 - Proven SOTA on MIRACL & MKQA benchmark datasets.
 - Reranker's cross-encoder precision removes "high-score low-relevance" false positives—which is crucial when long transcripts mention tangential topics.

12.2.7 Fallback & Governance Logic

- Hard-timeout triggers an automatic retry with the next-lighter model in the cascade.
- Every request/response pair, including router JSON is logged with SHA-256 request hash for reproducibility in future audits.

12.3 N-tier Architecture

The platform follows a classic 3-tier (Presentation/Business/Data) model plus two cross-cutting tiers – Security & Comms and an AI-specific RAG tier to keep the offline appliance modular, maintainable, and role-aware. Hardware can scale from a “basic PC” to a full rack-mount server, but typical deployments are sized for **5-10 concurrent** users per school. All tiers run inside the customer’s LAN; the device never requires internet connectivity.

12.3.1 Presentation Component

The Presentation Layer delivers a clean and intuitive interface for end users including teachers, students, and administrators.

- **Framework & Language:** Next.js (React) written in TypeScript for SSR/SPA flexibility.
- **State Management:** React Context/Hooks for local state; can promote to Zustand/Redux if multi-window sessions become heavy.
- **UI Features:** Claude-style conversational UI for teachers & students, additional widgets include an editable poster canvas (Fabric.js), PDF viewer, and role-based dashboards.
- **Routing:** Next.js file-system routes expose distinct modules (Inspection, Curriculum, Student Assistant, Marketing)

12.3.2 Business Logic Component

This layer houses the core processing, validation, RAG orchestration, and role-based operations.

- **Backend Framework:** Django (Python), we will leverage its robust admin, ORM, and RBAC middleware.
- **Domain Apps:** Separate Django apps (inspection, poster_gen, rag_chatbot, etc.) allow independent evolution and test pipelines.
- **RAG Orchestration:** Manages extraction, chunking, embedding, reranking, and prompt assembly before handing off to the local LLM.
- **Role Enforcement:** Central policy engine hides or reveals functionality per subscription plan and user role.
- **Offline Update Manager:** Admin UI triggers re-embedding or knowledge base rebuilds without external calls.

12.3.3 Data Access Component

This component connects the application logic with persistent and vectorized storage systems.

- **Database:** PostgreSQL stores structured data (user profiles, chat history, roles, vector index metadata).
- **Vector DB:** FAISS on-device for high-speed cosine/inner-product search; separate partitions for core knowledge base.
- **ORM:** Django ORM abstracts SQL while enabling migrations and field-level encryption.
- **Lifecycle:** PDF → text → chunks → embeddings → FAISS IDs stored back to Postgres.
- **Retention Guards:** Storage caps raise warnings, and block further uploads when capacity nears 45 GB, per stakeholder requirement.

12.3.4 Communication & Security Component

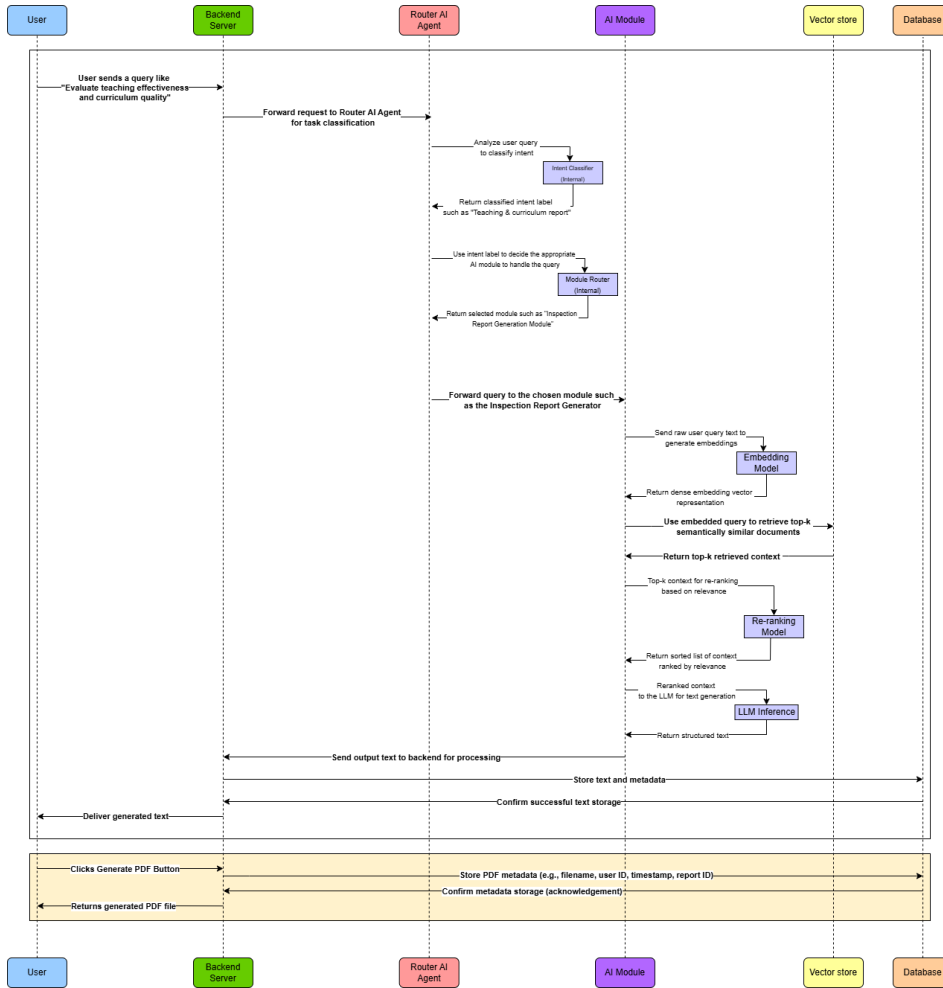
Since this system operates offline, security focuses on intranet protection, data integrity, and controlled access.

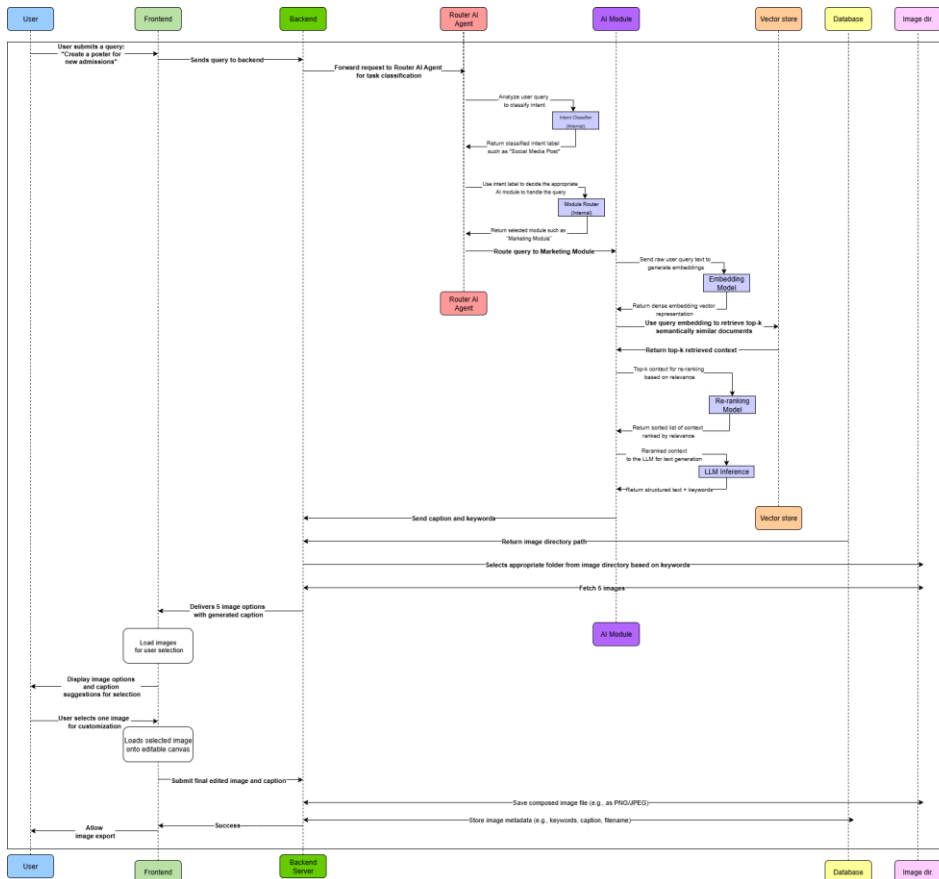
- **Reverse Proxy (Optional):** Nginx or similar can route frontend/backend requests securely in LAN.
- **Authentication:** JWT (or Django sessions) with granular RBAC; admins can audit any user's chat history for safeguarding.
- **Role-Based Access:** Permissions differ between Admins (e.g., updating knowledge base) and Users (e.g., chatbot interaction).
- **Intranet-only Exposure:** Device binds to a private IP; firewall rules block egress.
- **Logging:** Logins, file ingests, vector-store mutations, and LLM prompts all persisted for audit trails.

12.3.5 API Component

Provide a uniform, versioned interface that connects the Next.js frontend to back-end microservices in a secure, offline-first manner.

- **Framework:** Django REST Framework (DRF)
- **Authentication & Authorisation:** Short-lived JWT access tokens (15 min) plus refresh tokens (8 h) issued by the User Management Service. API Gateway verifies JWT, enforces scopes/roles
- **Core Endpoints:**
 - Admin: Create/update knowledgebase, upload PDFs
 - User: Query chatbot, get matched documents, generate posters
- **Versioning:** Future support for versioned APIs to maintain backward compatibility.





12.4 Microservices Architecture

Instead of deploying a monolithic application where all layers reside in a single codebase, the platform is decomposed into independent, domain-specific microservices. These services communicate via REST APIs, operate offline, and may maintain their own local data stores or access shared databases in a structured, secure way.

This design ensures modularity, fault isolation, and flexibility in development and scaling, even in an offline deployment.

12.4.1 Key Benefits

- **Modular Scalability:** Each domain runs in its own container; inspection or curriculum services can scale independently of poster generation.
- **Fault Isolation:** Failure of one system will not disturb the functionality and stability of the rest. For instance, a crash in PDF-rendering or embedding logic cannot cascade and take down chat or admin flows.
- **Domain-Centric Design:** Small, focused codebases let individual developers own a service end-to-end.
- **Hardware Optimization:** Compute-heavy inference containers can be scheduled on GPU nodes while lighter CRUD services stay on CPU-only hosts.

12.4.2 Core Microservices and Their Responsibilities

1. User Management Service

Responsibilities:

Authenticate users, issue/refresh JWTs, enforce RBAC, provide password-reset & audit-log export.

Features:

- JWT issuance & validation
- Role & permission CRUD
- User profile CRUD

2. Inspection Knowledgebase Service

Responsibilities:

Ingest inspection docs, clean & segment text, request embeddings from Inference, store in FAISS, expose /inspection/query RAG API.

Features:

- PDF ingestion & metadata extraction
- Publishes kb.uploaded, embed.done events
- Returns cited-chunk PDFs for chat

3. Curriculum & Lesson Planning Service

Responsibilities:

Maintain syllabus/grade docs, generate lesson plans, export PDFs.

Features:

- Schema-versioned document store
- Embeddings via Inference
- Chatbot planner
- Lesson plan PDF export

4. Student Learning Assistant Service

Responsibilities:

Manage personas, accept student prompts, retrieve teacher knowledge base, stream answers, export notes.

Features:

- Persona config per class
- Session uploads (e.g., homework) indexed temporarily
- Markdown / PDF note export

5. Poster & Campaign Service (Admission/Marketing)

Responsibilities:

Generates admission posters via AI-tag-matching or image generation.

Features:

- Keyword → asset tagging
- Fabric.js overlay editor
- Final image export

6. Document Generation Service

Responsibilities:

Convert HTML/Markdown to styled PDF, sign & return download URLs.

Features:

- Header / Footer templating
- Batched render jobs

7. Inference Service

Responsibilities:

Runs local LLMs for answering questions, generating text summaries, and embeddings.

Features:

- Ollama Backend
- Embedding
- Reranking
- Model Inference
- Persona conditioning

8. Config & Secrets Service

Responsibilities:

Central encrypted store for DB cred, model paths, feature flags; read-only API to other services.

Features:

- Version history & rollback

Responsibilities:

Allows admins to manage users, knowledgebases, and service configurations.

Features:

- Upload inspection / Curriculum datasets
- Tag / Image mapping for marketing
- Persona configuration for students

12.4.3 Communication and Integration Pattern

Service-to-Service Communication

- **Communication Protocols:**
 - All internal microservices communicate via **RESTful HTTPS** (<https://gateway.local>).
 - Ingress traffic terminates at the API Gateway; downstream calls will be encrypted and will perform mutual authentication—even on localhost.
- **Routing & Gateway:**
 - An **NGINX API Gateway** routes each frontend or inter-service request to the correct backend (e.g., chatbot, poster generation, knowledge-base).
 - **Service discovery:** Docker-Compose hostnames on single-box deployments
- **Asynchronous Communication:**
 - For decoupled workflows (e.g., embedding generation, PDF rendering) services publish events to a local bus, ensuring exactly-once delivery and replay—compatible with offline operation.
 - Uploads and generation endpoints must include an Idempotency-Key header; the Gateway discards duplicates.

Asynchronous Messaging

- **Event Handling:** Services emit and consume local events such as:
 - kb.uploaded
 - embed.done
 - poster.exported
 - pdf.renderedThese events trigger background jobs without blocking user actions, providing smoother UX and eventual consistency.
- **Message Queue System:** Redis Streams is the preferred queue for durability, consumer-group semantics, and at-least-once guarantees. When Redis is unavailable, a lightweight SQLite-backed queue can provide minimal offline resilience.

12.4.4 Security and Compliance (Offline-Oriented)

Authentication & Authorization

- **JWT-Based Authentication:** The User Management Service issues short-lived JWT access tokens (15 min) plus refresh tokens (8 h) after successful login.

- **Role-based access control (RBAC):** Roles like admin, teacher, and student are checked at each microservice. Scopes are embedded in the JWT and verified by the API Gateway.
- **Token verification:** Every internal service validates the JWT signature and expiration; expired or tampered tokens are rejected and details are logged.
- **Secrets management:** All passwords, DB credentials, and model paths are stored in the Config & Secrets Service and encrypted.

Transport-Level Security (Internal)

- All REST and gRPC calls will use **TLS 1.3 with mutual authentication**; certificates are issued by the on-box Certificate Authority.
- Downstream traffic remains encrypted and authenticated.

Auditing & Logging

- The API Gateway injects an X-Correlation-ID header into every request; the ID propagates through REST, gRPC, and Redis Streams.
- Logs (requests, errors, service actions) are stored locally in append-only files on the hardware's SSD in append-only mode.

12.5 Method of Development

The system will be developed in-house by the company's engineering team using industry-standard software engineering practices. The development process is designed to ensure code quality, modularity, and maintainability with a focus on delivering a robust product suitable for offline deployment in school or institutional environments.

- **Development Approach:** The platform will be built using an Agile methodology, with work organized into iterative sprints. Regular planning, stand-ups, reviews, and retrospectives will drive progress and adaptation based on evolving requirements.
- **Version Control:** All source code will be managed using Git and hosted on a secure, private repository. Feature branches, pull requests, and issue tracking will be used to manage collaboration and changes.
- **Code Quality & Reviews:** Peer reviews will be enforced for all major contributions. Automated tools like ESLint, Prettier, and Pylint will ensure codebase consistency and static code quality checks.
- **Testing:**
 - Unit Testing using frameworks like Jest (JavaScript/TypeScript) and PyTest (Python) will be employed to validate individual modules.
 - Integration Testing will verify that different services work together across the frontend, backend, and AI components.
 - Manual and automated testing will be done in staging environments to simulate offline deployment scenarios.
- **Offline Packaging and Deployment:** Once development is complete, the platform will be packaged for offline deployment, including:

- All models and dependencies bundled.
 - Docker or virtual environment setup scripts
 - Admin UI for local setup, configuration, and model control
 - No external API or internet connectivity required post-deployment
- **Documentation:** Comprehensive technical documentation will be prepared covering API contracts, system architecture, model setup, and deployment instructions. An offline user guide/manual will also be created for school administrators and teachers.

12.6 High-Level Technical Design

- Users access the appliance through a responsive web interface built with Next.js (React + TypeScript).
- The browser exchanges data with the backend exclusively over HTTPS to `https://gateway.local`, where the NGINX API Gateway:
 - Validates JWT access tokens and roles,
 - Injects an X-Correlation-ID,
 - Applies rate-limits and forwards the request to the appropriate microservice.
- The backend consists of containerised Django microservices.
- For compute-intensive tasks (embeddings, text generation, reranking) these services call the Inference Service via gRPC streams, while traditional CRUD endpoints stay on REST.
- RBAC, quota checks, and workflow orchestration (e.g., retrieval-augmented generation, poster creation) are enforced in this tier.
- Persistent data flows through the Data Access Layer:
 - **PostgreSQL:** structured data (users, chat history, audit events, vector-ID metadata).
 - **FAISS:** semantic index for document chunks; IDs & collection info stored back in PostgreSQL.
 - **Config & Secrets Service:** encrypts DB creds, model paths.
- Services integrate in two ways:
 - **Synchronous:** REST or gRPC with mTLS for low-latency reads/writes.
 - **Asynchronous:** Redis Streams topics such as `kb.uploaded`, `embed.done`, `pdf.rendered`; consumer groups guarantee exactly-once processing, enabling decoupled jobs like embedding generation or PDF rendering.
- All AI models reside on the appliance in GGUF or Safetensors format, running on GPU if present or CPU fallback, ensuring strict data residency and compliance with school privacy regulations. The design contains no external API calls, allowing complete operation in a disconnected LAN.

The technical design and selected tech stack are subject to refinement based on developer experimentation and evolving project requirements. The architecture aims to balance complexity with the need for scalability, security, and performance.

13. Hardware Requirements

14. Acceptance Criteria

The Acceptance Criteria define the measurable conditions that the system must meet to ensure the system's functionality aligns with business requirements. These criteria serve as a benchmark for validating the system's usability, performance, and compliance with industry standards.

a. General System Behaviour

- The system should allow users to select a specific use case at the start of the conversation through a configurable menu or prompt suggestion.
- The system should support multilingual input/output (English and Arabic), with user preference toggle available during the session.
- The system should guide the user with predefined prompt suggestions and quick action buttons tailored to each selected use case.
- The system should allow voice input and generate accurate transcripts for prompt submission, along with the ability to listen to AI responses.
- The system should ingest uploaded files and use them to enrich conversational context.
- The system should store and retrieve prior user interactions, showing a history of generated outputs for each user session.
- The system should maintain role-based permissions to restrict or enable features such as editing, downloading, and content generation.
- The system should gracefully handle model handoff (model chaining) between use cases without breaking conversational flow.
- The system should provide feedback when queries fall outside the supported scope and suggest relevant use case actions instead of failing silently.
- The system should operate in offline mode, with all critical features available when disconnected from the internet.
- The system should allow the Super Admin to configure which models are used per use case, and whether chaining is enabled between modules.
- The system should prevent unsafe or inappropriate user inputs via built-in moderation filters, especially for student-facing flows.
- The system should prevent unsafe and inappropriate user inputs via built-in moderation filters, especially for student-facing flows.

Commented [SA17]: for text yes, for voice - only English

b. Use Case 1: AI Driven School Inspections

- The system should allow inspectors or school leaders to initiate a new inspection by selecting a predefined framework and prompt set.
- The system should support uploading of previous or internal reports as context to improve the relevance of generated insights.
- The system should generate inspection reports using pre-mapped school data and uploaded documentation, aligned with regulatory framework structure.
- The system should render the generated report in a canvas layout, with options to listen, download, or regenerate the report.
- The system should allow the user to ask contextual questions about the generated report and receive specific, evidence-backed responses.

- The system should allow the user to customize the layout of the report before final export.
- The system should support one-click export of the final report in PDF format, reflecting all edits and customizations.
- The system should enable saving generated reports to a central report library accessible based on user roles (Inspector, Leadership).
- The system should not allow real-time collaboration on reports but must ensure that role-based users can view or edit as permitted.
- The system should log all edits, exports, and conversations related to an inspection in the audit trail for traceability.

Commented [SA18]: customizations to the templates will be a part of a one time config service before deploying the system to the school, post-deployment if they want to change the template - they will need to contact support and request for it after which it will be developed

c. Use Case 2: Curriculum & Lesson Planning

- The system should allow educators to browse, search, and manage subject-specific content through a centralized Curriculum Content Repository.
- The system should assist teachers in generating lesson plans aligned with selected curriculum standards through conversational interaction.
- The system should support multi-step dialogue for designing lessons, including goals, learning outcomes, resources, and activities.
- The system should allow teachers to generate quizzes, assignments, and supplementary materials based on lesson objectives.
- The system should enable teachers to define or adjust lesson templates and save them to the repository for future reuse.
- The system should allow educators to create personalized learning pathways for students based on learning goals or challenges.
- The system should support teacher-driven content uploads for inclusion in lesson flows or recommendations.

Commented [SA19]: this will be possible only if the person with the necessary permissions to update the knowledge base performs this operation (for eg. HOD)

d. Use Case 3: Student Learning Assistant

- The system should allow students to ask academic or study-related questions via text or voice in a simplified chat interface.
- The system should generate personalized, context-aware responses based on uploaded classroom materials and system-provided learning objectives.
- The system should allow teachers to predefine the AI assistant's behaviour (tone, persona, style of response) based on grade or learning style.
- The system should allow students to receive follow-up questions, topic summaries, and explanation breakdowns in a conversational flow.
- The system should allow students to upload classroom materials like worksheet images, textbook sections for contextual support, with teacher-controlled limits.
- The system should enforce strong moderation for student prompts, rejecting inappropriate or off-topic questions and guiding students back on track.
- The system should log all student interactions and maintain an auditable record visible only to school leadership or safeguarding officers.
- The system should restrict access to only supervised; in-class use preventing remote usage.

e. Use Case 4: Admissions & Marketing

- The system should allow admissions or marketing users to initiate a conversation to review admissions workflows, applicant insights, or generate campaigns.

- The system should support conversational intake of applicant data to evaluate profiles based on pre-defined school criteria or historical benchmarks.
- The system should allow users to ask questions like “Show me top applicants from last year” or “Evaluate this applicant’s fit” and receive scored or narrative insights.
- The system should allow the user to specify a platform (e.g., Instagram, LinkedIn) for which a campaign asset should be generated, using either conversational prompt or selection UI.
- Once generated, the system should show the campaign image in line with edit and regenerate buttons.
- The system should allow basic image editing (text, font, colour, style, size, bullet formatting) within a lightweight editor. Image type changes must trigger regeneration.
- The system should retain the original prompt context during edits to avoid model drift or hallucination during re-generation.
- The system should allow users to request marketing performance summaries, presented as visual insights or brief analytics-style reports.
- ~~The system should generate foundational enrolment forecasts or churn predictions based on historical admissions and SIS data.~~

f. Admin Configurations

- The system should restrict Admin Configuration access to authorized users only.
- The system should allow admins to customize branding elements, including:
 - Platform logo upload.
 - Primary and secondary ~~color~~colour palette for document theming.
 - ~~Footer text and e~~Optional institution-specific disclaimers. (TBD)
- Admin users should be able to:
 - Create, edit, and deactivate user accounts for teachers, students, and other admins.
 - Assign and modify roles (e.g., Teacher, Admin, Student) with corresponding access levels.
 - Reset passwords or enforce password policies.
 - Role-based permissions should control access to all configuration features.
 - The system should validate unique email/username and disallow role conflict errors (e.g., assigning both student and admin roles).
 - Deactivated users should not be able to log in or access platform features.
- ~~The system should provide a configuration toggle to:~~
 - Daily/monthly active users by role.
 - Top used features (e.g., lesson plan generation, student help).
 - AI usage trends and average response lengths.
 - Export of usage analytics as CSV or PDF should be supported.
- Admins should be able to:
 - Upload, tag, and organize documents for Inspection, Curriculum, or Student Assistant modules.
 - Control visibility of KB items per status.
 - Manage knowledgebase active/inactive states.
- The system should:
 - Automatically process uploaded files into embeddings and store metadata.
 - Notify the admin if the document is malformed or too large.

- Prevent duplicate uploads of the same file (by content hash).
- Admin should be able to archive or delete any KB, with confirmation prompts and metadata logging.
- Admins should be able to view a list of AI responses flagged due to:
 - Student self-reporting.
 - Teacher moderation.
 - Automated moderation rule triggers (if configured).
- Each flagged item should include:
 - Timestamp
 - User role
 - Input query
 - AI response
 - Flag type (Inappropriate, Inaccurate, Out-of-scope)
 - Any message by the user
- Admins should be able to export flagged logs for school audits.
- Admin should be able to configure:
 - School-wide AI response tone and persona conditioning (e.g., formal, instructional).
 - Enable/disable modules (e.g., deactivate Marketing if not needed).
 - All configurations should be stored securely and be update-safe.
 - Unauthorized users should not be able to view or alter configuration screens.

15.—Product Future

This section captures outstanding questions and clarifications that require input from the client. Addressing these queries is essential to finalising system behaviour, feature design, or configuration choices before the development phase.

- a. Should session memory be persistent across logins or reset per session?
- b. Who owns and governs the school-uploaded knowledgebase? Can it be centrally reviewed?
- c. Should we provide user-configurable KPIs or only support KHDA's fixed metrics?
- d. What format constraints (if any) are imposed on inspection reports by authorities?
- e. Will the system support multi-device login for students within the same school day?
- f. Can schools apply their own filtering logic or moderation thresholds?
 - If yes?
 - Then schools will be able to configure moderation logic via settings.
 - Types of moderation can include keyword blocking, content classification, toxicity thresholds.
 - These filters will be enforced by a prompt moderation middleware before reaching the AI model.
 - Admins can set custom thresholds, upload disallowed keywords, and view flagged queries via logs.
- g. Are there target KPIs for conversion rates, or should the AI suggest baseline benchmarks?

Commented [SA21]: to be removed and answers received by Riza to be inculcated within the current SRS